

RFC: Password-Based Key Management for the Encryption VFD

Clay Carper clay.carper@lifeboat.llc

In the context of the Encryption VFD, key management is an essential component of providing a comprehensive data security solution. This document describes the proposed password-based key management architecture used to create, protect, recover, and manage Data Encryption Keys (DEKs). As an initial implementation, a local, operating system-agnostic key management mechanism is proposed in which a randomly generated DEK is encrypted, or wrapped, using a password-derived Key Encryption Key (KEK). At runtime, the user's password is used to reconstruct the KEK, authenticate the DEK+ object, recover the DEK, and store it within a `libgcrypt` secure memory pool for use by the Encryption VFD. Although the architecture is designed to accommodate additional authentication mechanisms in the future, the initial implementation supports only password-derived KEKs. Definitions of the terminology used throughout this document are provided in Appendix A – Glossary.

Table of Contents

1 Introduction4

2 Key Management From a User Perspective5

2.1.1 Selecting a Key Management Workflow6

2.1.2 Creating an Encrypted HDF5 File with Terminal-Based Input6

2.1.3 Creating an Encrypted HDF5 File Without Terminal-Based Input8

2.1.4 Opening an Encrypted HDF5 File with Terminal-Based Input9

2.1.5 Opening an Encrypted HDF5 File Without Terminal-Based Input10

2.1.6 Password Management Overview.....11

2.1.7 Change password with Terminal-Based Input.....11

2.1.8 Change password without Terminal-Based Input12

2.1.9 Add a password with Terminal-Based Input.....13

2.1.10 Add a password without Terminal-Based Input14

2.1.11 Remove a password with Terminal-Based Input.....15

2.1.12 Remove a password without Terminal-Based Input15

2.1.13 Migrating Between Supported DEK Handling Mechanisms16

3 Implementation Outline17

3.1 High-Level Usage Description17

3.2 DEK+ Format Summary.....19

3.3 Password Usage.....19

3.3.1 Terminal-Based Consent and Password Prompt20

3.3.1.1 Layer 1 — Plain Terminal Fallback.....20

3.3.1.2 Layer 2 — Boxed In-Terminal View22

3.3.1.3 Layer 3 — Extended Platform-Specific Secure Input Backend23

3.3.1.4 Terminal Echo Suppression24

3.3.1.5 Password Confirmation and Empty-String Passwords24

3.3.1.6 Interactive and Non-Interactive Input25

3.3.1.7 Initial Implementation Scope25

3.3.1.8 Failure and Cleanup Behavior26

3.4 User-Interaction, Build Capability, and Policy Model27

3.4.1.1 Effect on Build Options28

3.4.1.2	Effect on Runtime Operation Flags	29
3.4.1.3	Effect on DEK+ Object Flags	30
3.5	Usage with the Encryption VFD	31
3.6	H5keytools Companion Utility	32
3.6.1	Purpose	33
3.6.2	Supported Operations	33
3.6.2.1	Create DEK	33
3.6.2.2	Create DEK+ Object	33
3.6.2.3	Validate DEK+ Object	34
3.6.2.4	Change Password	34
3.6.2.5	Remove Password	34
3.6.2.6	Add Password	34
3.6.2.7	Update KDF Parameters	35
3.6.2.8	Future Format Migration	35
3.6.3	Transferring Between Supported DEK Handling Mechanisms	35
3.6.3.1	Convert from Secure to Secure	36
3.6.3.2	Convert from Insecure to Secure	36
3.6.3.3	Convert from Secure to Insecure	36
3.6.4	Build-Time Protection	37
4	Theory of Operations	37
4.1	Create a New DEK+ Object	37
4.2	Interacting with an Existing Encrypted File	38
4.3	Rotating the KEK using h5keytools	39
5	Technical Details	39
5.1	Encryption VFD Overview	39
5.2	DEK+ Format	40
5.2.1	DEK+ Object Layout	40
5.2.2	DEK+ Header Structure	43
5.2.3	DEK+ Header Flags	47
5.2.4	Serialization and Endianness	48
5.2.5	Cryptographic Construction	48

- 5.2.6 Secret and Non-Secret Material50
- 5.3 Code Components50
 - 5.3.1 Public LFBT_keymgmt Operations.....51
 - 5.3.2 Runtime Policy and Operation Flags53
 - 5.3.3 Internal HDF5/VFD Control Dispatch.....55
 - 5.3.4 Internal High-Level DEK+ Operations56
 - 5.3.5 Omnibus Input and Output Structures.....57
 - 5.3.6 LFBT_keymgmt I/O Abstraction66
 - 5.3.7 Internal DEK+ Read/Write Helpers.....74
 - 5.3.8 Internal Cryptographic Helpers76
 - 5.3.9 Password and Policy Helpers.....77
- 5.4 Call-Stack Outlines78
 - 5.4.1 Creating a New DEK+ Object78
 - 5.4.2 Unlock Existing DEK+ Object.....79
 - 5.4.3 Rekey Workflow (h5keytools) (Password Change/Removal)79
- 5.5 Testing Plan80
- Appendix A – Glossary81

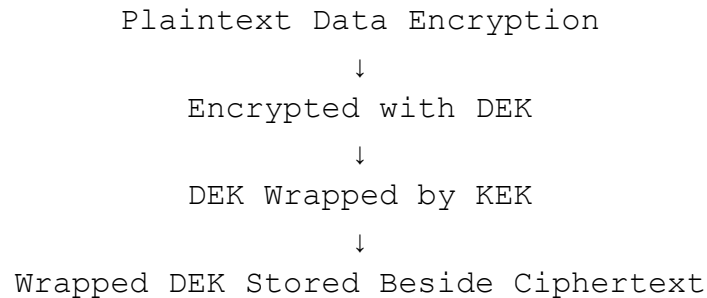
1 Introduction

In the context of the Encryption VFD, key management is an essential component for providing users with a comprehensive data security solution. As a first step, a local, operating system-agnostic key-protection mechanism will be utilized. Under this model, a random Data Encryption Key (DEK) is generated locally and then encrypted, or wrapped, using a password-derived Key Encryption Key (KEK). At runtime¹, the user’s password is then used to recover the DEK, which is then stored in a `libgcrypt` secure memory pool and used by the Encryption VFD. While this solution provides a framework appropriate for utilizing other authentication methods, at the time of writing only password-derived KEKs will be supported. Concerning the terminology used throughout this document, definitions are made available in Appendix A – Glossary.

¹ In environments where an interactive terminal is unavailable, a caller may use an empty-string password when the application build capabilities and runtime policy permit it. This allows the wrapped DEK to be recovered without interactive password entry. This mode is intended primarily for automated testing, batch processing, or HPC environments where interactive authentication is impractical. Because an empty-string password provides no password entropy, it should be used only in trusted environments.

2 Key Management From a User Perspective

In order to properly utilize the proposed password-based key management solution, it is crucial to understand the underlying principle that inspired this approach, envelope encryption. Envelope encryption refers to the process of wrapping an encryption key using a different, secondary encryption key. Simply stated, the two keys used are referred to as a Data Encryption Key (DEK), which is used to encrypt data (i.e., the key used to secure the content of an encrypted HDF5 file) and a Key Encryption Key (KEK), which is the encryption key used to encrypt a DEK. In order to create a clean nomenclature, a DEK that is encrypted using a KEK is often referred to as a wrapped DEK. Generally speaking, the underlying encryption model is as follows:



The KEK is never stored directly. Instead, it is reconstructed whenever needed from the user's password and the information stored in the DEK+ object. The recovered KEK is then used to unwrap the DEK, after which the KEK is discarded while the DEK remains available for use by the Encryption VFD. What this means in practice is that the password-protected DEK is typically never directly exposed to the user, reducing the amount of knowledge required to work with an encrypted HDF5 file. Further details relating to the creation and usage of these principles with respect to an encrypted HDF5 file are discussed later in this section.

The information required to recover the DEK is stored within a serialized binary object referred to throughout this document as a DEK+ object. Each DEK+ object conforms to the DEK+ format defined in this document. Rather than storing the cleartext DEK (unless configured otherwise) or the user's password, the DEK+ object stores the wrapped DEK together with the cryptographic metadata required to reconstruct the KEK, verify the integrity of the stored information, and recover the DEK during future operations. For the initial implementation, the DEK+ object is expected to reside within a reserved page of the encrypted HDF5 file, allowing the Encryption VFD to automatically locate the information needed to recover the DEK whenever the file is opened. Because a password-protected DEK+ object contains only encrypted key material and supporting metadata, users are not expected to inspect or modify its contents during normal operation.

Envelope encryption also provides a natural mechanism for accommodating password changes. In this context, a password change will result in the creation of a new KEK. This process is commonly referred to as a key rotation operation. Under the principles of envelope encryption, the underlying DEK remains constant, allowing for the periodic rotation of KEKs. When a KEK is rotated, the underlying data need not be re-encrypted [2]. Instead, the associated DEK is simply unwrapped with the old KEK, then rewrapped with the new KEK.

2.1.1 Selecting a Key Management Workflow

Users select the desired key management behavior through the Encryption VFD configuration, the h5keytools companion utility, or the corresponding public API. Users are not expected to construct internal operation flags or modify the flags stored within the DEK+ object. These values are selected and maintained automatically by the implementation based on the requested operation.

The capabilities available to the user may depend on how the Encryption VFD and h5keytools utility were built. Security-sensitive functionality, such as empty-string password support, cleartext DEK storage, cleartext DEK export, or creation of an external DEK+ copy, may require the corresponding build option to be enabled. If the user requests an operation that is not available in the installed build, the operation will fail with an error identifying the unavailable capability. For interactive workflows, h5keytools or the Encryption VFD collects the required password and confirmation input through the terminal. For non-interactive workflows, the calling application supplies the required values programmatically. Both workflows invoke the same underlying implementation and produce equivalent cryptographic results when provided with the same inputs and policies.

The use cases discussed in the remainder of this section are intended to reflect how the user is expected to interact with the password-based key management solution presented in this document. For each operation outlined in this section, it is assumed that the user has determined whether to use an interactive or non-interactive workflow and has selected a workflow supported by the capabilities present in the installed build.

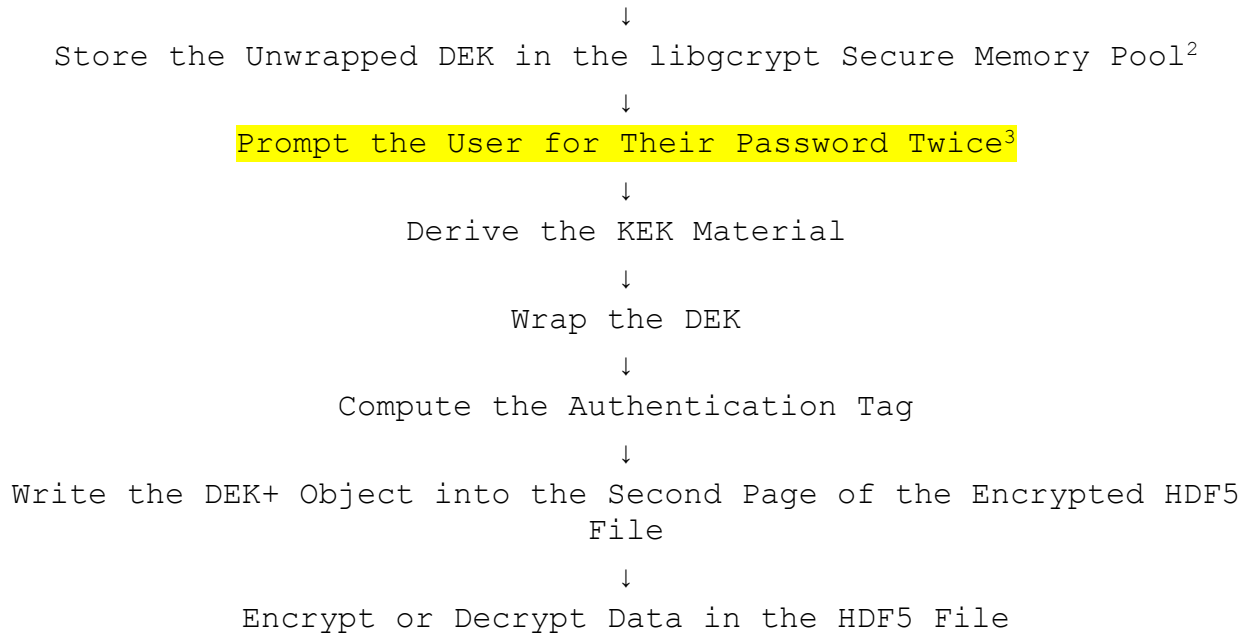
2.1.2 Creating an Encrypted HDF5 File with Terminal-Based Input

- Creating a new encrypted HDF5 file with DEK+ support will automatically generate a new DEK+ object and store the recovered plaintext DEK within the libgcrypt secure memory pool for immediate use by the Encryption VFD. Unless the user explicitly supplies an existing 32-byte DEK, a new cryptographically secure random DEK is generated.
- If the user has a desired DEK or a preferred method of key generation, they may optionally supply a 32-byte DEK through the Encryption VFD configuration string. Doing so is generally discouraged, as reusing the same DEK across multiple encrypted HDF5 files increases the impact of a DEK compromise.
- The workflow assumes that interactive terminal input is available. Consequently, the user is prompted to enter their password during file creation, allowing the Encryption VFD to derive the KEK without exposing the password through the configuration string.
- The procedure below illustrates the general sequence of operations performed when creating a new encrypted HDF5 file using interactive password entry. Highlighted components indicate where user interaction is expected to occur.

User Calls H5Fcreate() to Create an Encrypted HDF5 File



Generate a Random DEK (or Use the User-Provided 32-byte DEK)



Once the encrypted HDF5 file has been created, the recovered DEK remains resident within the libgcrypt secure memory pool for the lifetime of the open file. Consequently, the user is prompted for their password only during file creation and is not required to repeatedly authenticate as additional encrypted file content is accessed. The password-derived KEK exists only long enough to protect the DEK and is securely erased immediately afterward.

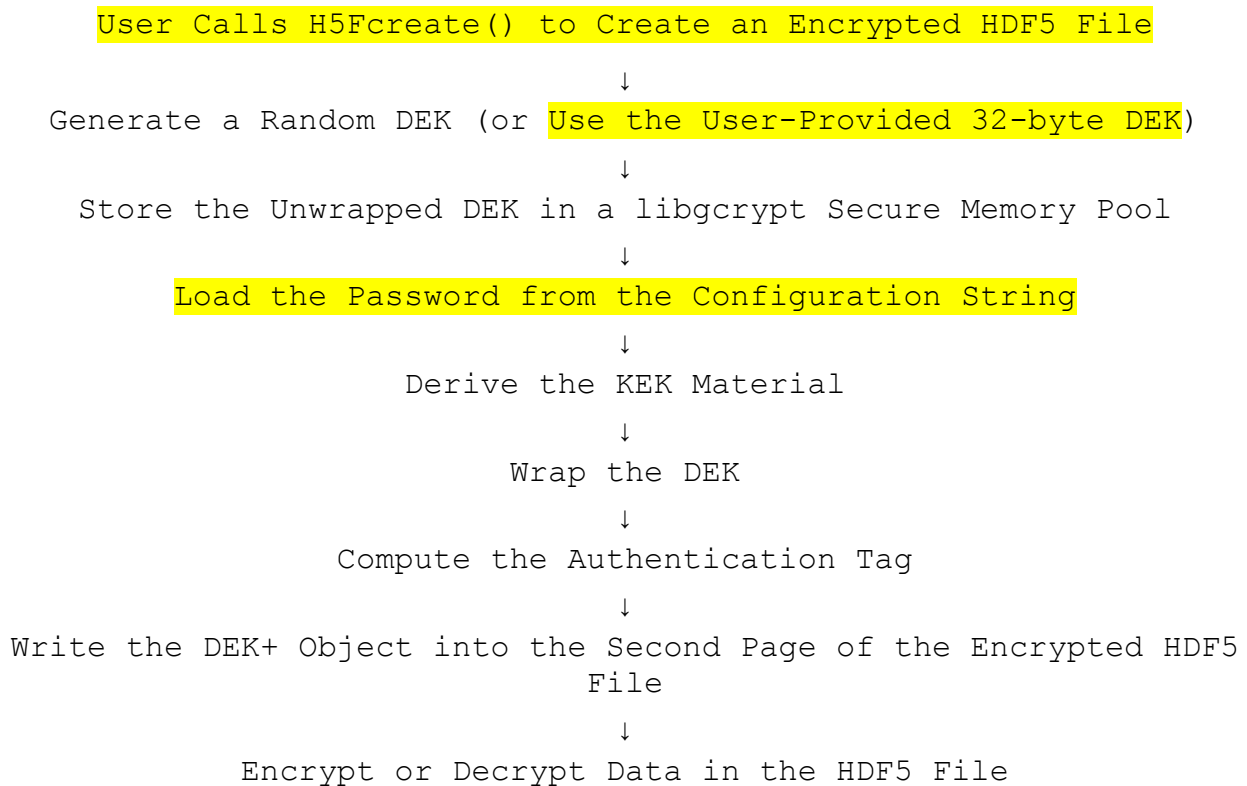
2.1.3 Creating an Encrypted HDF5 File Without Terminal-Based Input

- This workflow performs the same DEK generation and DEK+ creation process described in Section 2.1.2, except the password is provided programmatically rather than through an interactive terminal prompt.
- If the user has a desired DEK or a preferred method of key generation, they may optionally supply a 32-byte DEK through the Encryption VFD configuration string. Doing so is generally discouraged, as reusing the same DEK across multiple encrypted HDF5 files generally increases the risk associated with a compromised DEK.

² The unwrapped DEK is retained in the libgcrypt secure memory pool for the lifetime of the open encrypted HDF5 file. During an orderly file close or error-cleanup path, the owning component releases the secure memory allocation. An abrupt process failure may prevent application cleanup handlers from executing; secure erasure cannot be guaranteed after every crash.

³ The user is prompted for to enter the password twice. If the entries differ, the operation reports a mismatch and requests a new password and confirmation. When permitted by the application build capabilities and runtime policy, pressing **Enter** for both entries selects an empty-string password (""). This representation retains KEK-based wrapping and HMAC authentication, but the derived KEK has no password entropy and can be reconstructed by anyone possessing the DEK+ object. It should therefore be used only when interactive password entry is undesirable and a cleartext DEK representation is not required.

- The user may use a trivial password by passing "" rather than passing a non-trivial password (i.e., "mykepasswordgoeshere"). Because the password is provided directly through the configuration string, it exists in ordinary application memory before KEK derivation. Consequently, this workflow should be reserved for trusted systems or environments where interactive authentication is unavailable.
- The procedure below illustrates the general sequence of operations performed during non-interactive DEK+ creation. Highlighted components indicate where application-provided input is consumed.



Although the password is provided programmatically rather than entered through the terminal, the resulting cryptographic protection is identical to the interactive workflow. The only practical difference is the mechanism by which the password is provided to the Encryption VFD. Because the password exists in cleartext within application memory before key derivation, the workflow should be reserved for trusted systems or environments where interactive input is unavailable.

2.1.4 Opening an Encrypted HDF5 File with Terminal-Based Input

- Opening an encrypted HDF5 file recovers the existing DEK represented by the DEK+ object rather than generating a new DEK. Once recovered, the DEK remains resident within the libgcrypt secure memory pool for the lifetime of the open HDF5 file.

- Because the DEK+ object is stored together with the encrypted HDF5 file for the initial implementation, users typically specify only the encrypted HDF5 file during normal operation. Future implementations may additionally support external DEK+ objects.
- This workflow assumes that interactive terminal input is available. The user is prompted for their password only once when the file is opened, after which subsequent encryption and decryption operations proceed without further authentication.
- The procedure below illustrates the general sequence of operations performed when opening an encrypted HDF5 file using interactive password entry. Highlighted components indicate where user interaction occurs.

Open the Encrypted HDF5 File with H5Fopen()



Load the DEK+ Object from the Second Page of the Encrypted HDF5 File



Prompt the User for Their Password



Invoke the Password-Based KDF



Derive the KEK



Verify the Authentication Tag



Recover the DEK



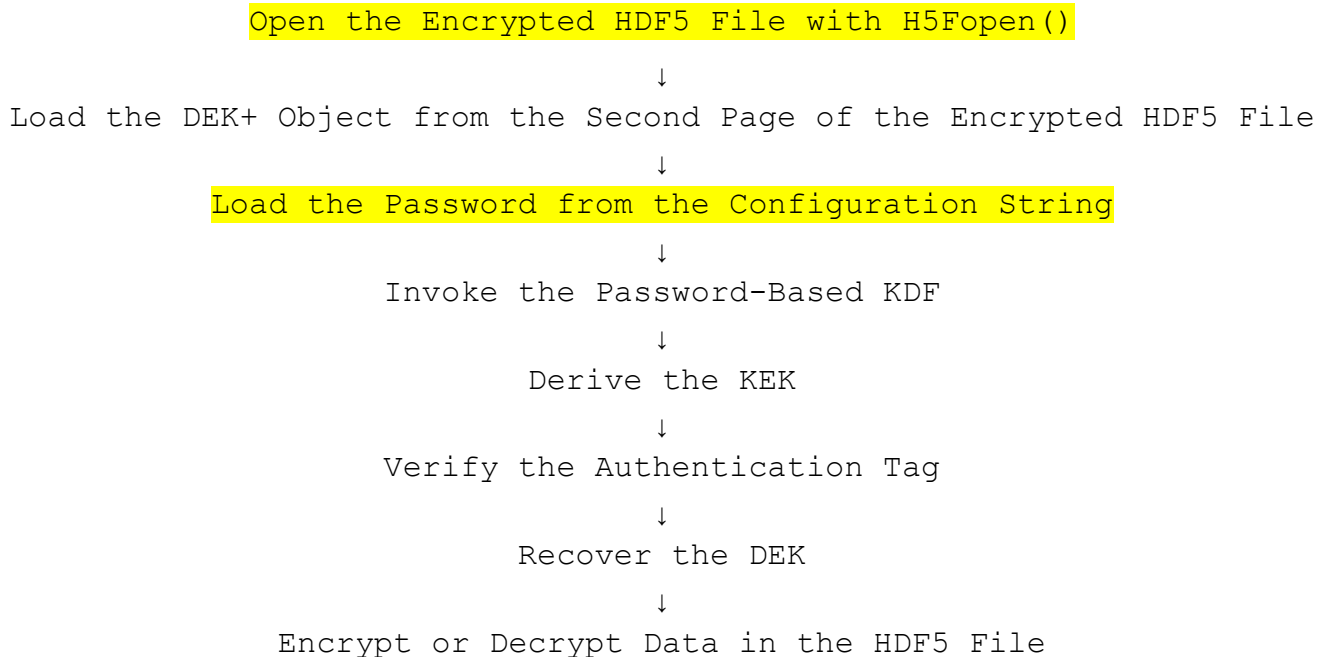
Encrypt or Decrypt Data in the HDF5 File

Successful authentication results in recovery of the original DEK rather than generation of a new encryption key. Once recovered, the DEK remains resident within the `libgcrypto` secure memory pool for the lifetime of the open HDF5 file, allowing the Encryption VFD to perform subsequent encryption and decryption operations without repeatedly requesting the user's password.

2.1.5 Opening an Encrypted HDF5 File Without Terminal-Based Input

- This workflow performs the same DEK recovery process described in Section 2.1.4, except the password is provided programmatically rather than through an interactive terminal prompt.
- As with the interactive workflow, the Encryption VFD automatically locates the associated DEK+ object and recovers the existing DEK without requiring the user to directly manipulate DEK+ metadata.

- The user will provide their desired password as part of the Encryption VFD configuration string. The user may use a trivial password by passing "" rather than passing a non-trivial password (i.e., "mykekpasswordgoeshere"). Because the password is provided through the application configuration or API, it exists in ordinary application memory prior to KEK derivation. Consequently, this workflow should be reserved for trusted environments where interactive authentication is impractical or unavailable.
- The procedure below illustrates the general sequence of operations performed during non-interactive DEK recovery. Highlighted components indicate where application-provided input is consumed.



From the perspective of the Encryption VFD, this workflow is functionally equivalent to the interactive recovery process. The only difference lies in how the password is provided. Because the password is available in cleartext to the calling application before KEK derivation, users should carefully evaluate the security implications before selecting this workflow.

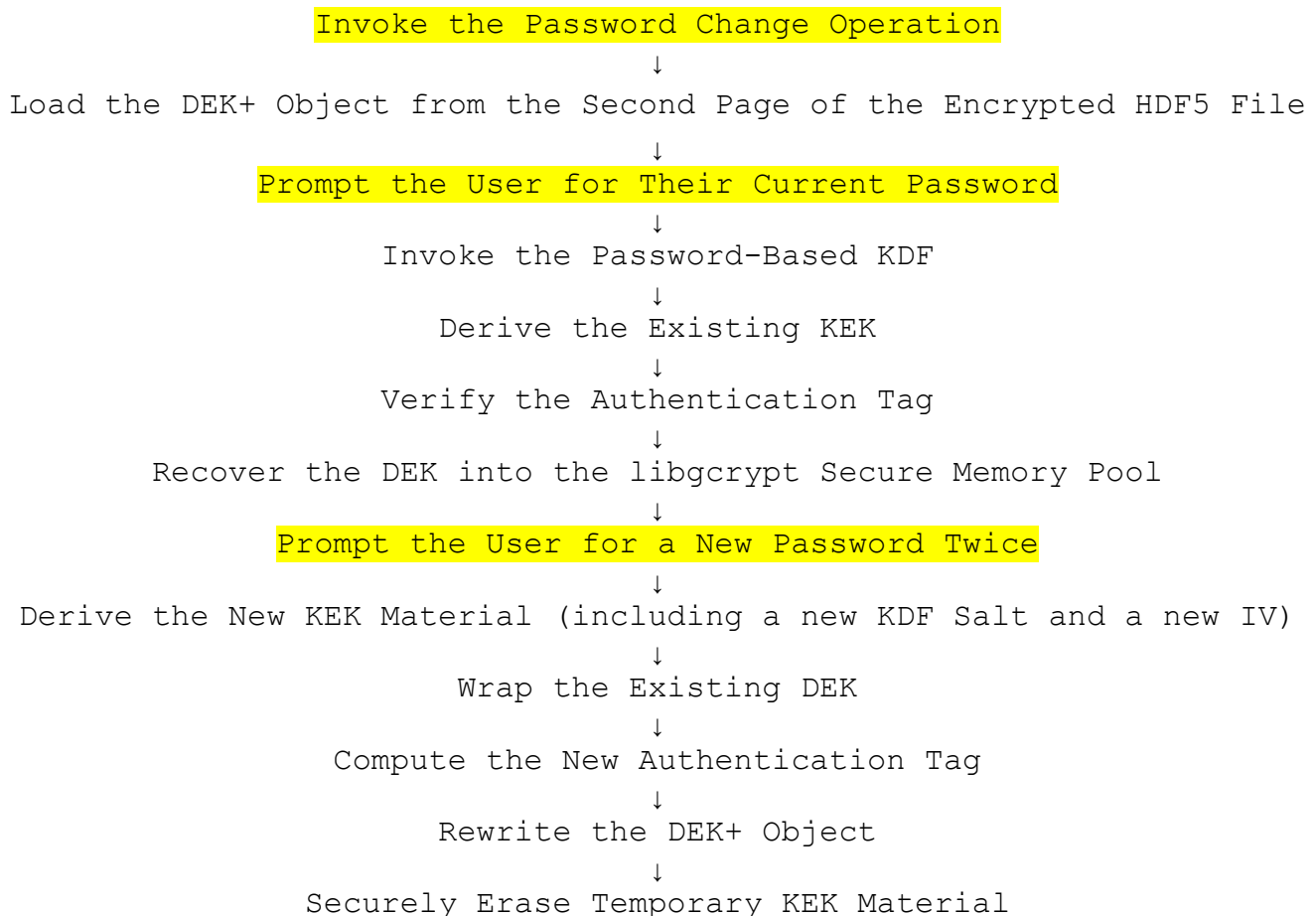
2.1.6 Password Management Overview

The password-management operations described in Sections 2.1.7 through 2.1.12 may be performed either through the `h5keytools` companion utility or through the corresponding public `LFBT_keymgmt` API. Both interfaces invoke the same underlying implementation and therefore provide identical functionality and cryptographic behavior. Throughout the workflows below, terminal-based examples reference `h5keytools` for clarity, while non-terminal examples illustrate the equivalent programmatic API usage.

It is important to distinguish between changing a password and removing a password. Changing a password always preserves password-based protection by replacing one password-derived KEK with another. If a user wishes to continue using password-based protection while adopting an empty-string password, the **Change Password** operation should be used. In contrast, the **Remove Password** operation disables password-based protection entirely by removing the KEK and storing the DEK in cleartext within the DEK+ object. These operations therefore have different security properties and are intentionally treated as separate workflows.

2.1.7 Change password with Terminal-Based Input

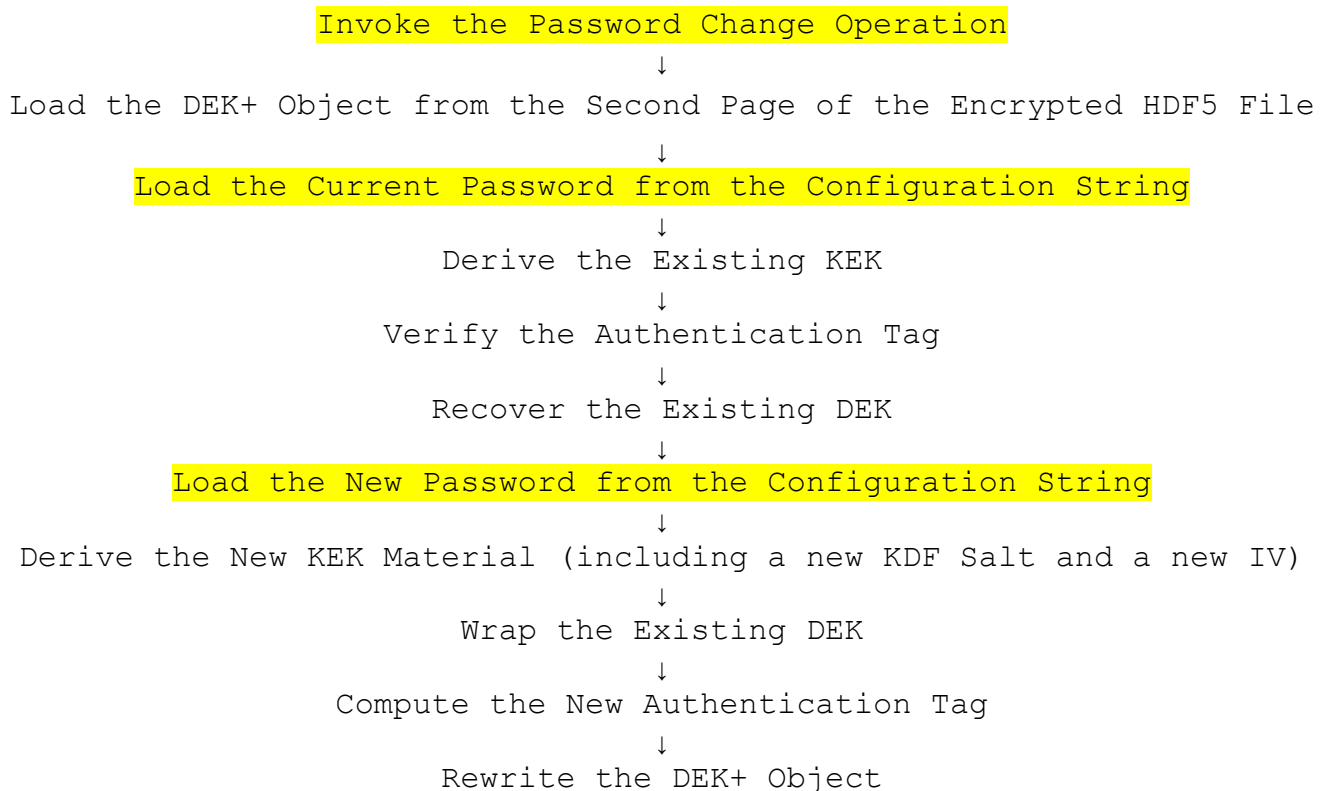
- Changing the password does not generate a new Data Encryption Key (DEK). Instead, the existing DEK is recovered using the current password and immediately re-protected using a newly derived Key Encryption Key (KEK). This operation is commonly referred to as KEK rotation.
- Since the DEK itself remains unchanged, the encrypted contents of the HDF5 file do not need to be re-encrypted. Only the DEK+ object is rewritten with newly generated cryptographic material.



Although the password changes during this operation, the underlying DEK remains unchanged. Consequently, the encrypted HDF5 data do not require re-encryption. Instead, only the password-derived KEK, KDF salt, initialization vector, wrapped DEK, and authentication tag are regenerated and written back to the DEK+ object. This significantly reduces the cost of changing passwords, regardless of the size of the associated encrypted HDF5 file.

2.1.8 Change password without Terminal-Based Input

- This workflow performs the same KEK rotation operation as described in Section 2.1.7, except the existing and replacement passwords are provided programmatically rather than being collected through an interactive terminal prompt.
- Password-management operations may be invoked either through the public LFBT_keymgmt API or by h5keytools, both of which share the same underlying implementation and produce identical DEK+ output.
- The procedure below illustrates the general sequence of operations performed during a non-interactive password change. Highlighted components indicate where application-provided input is expected.

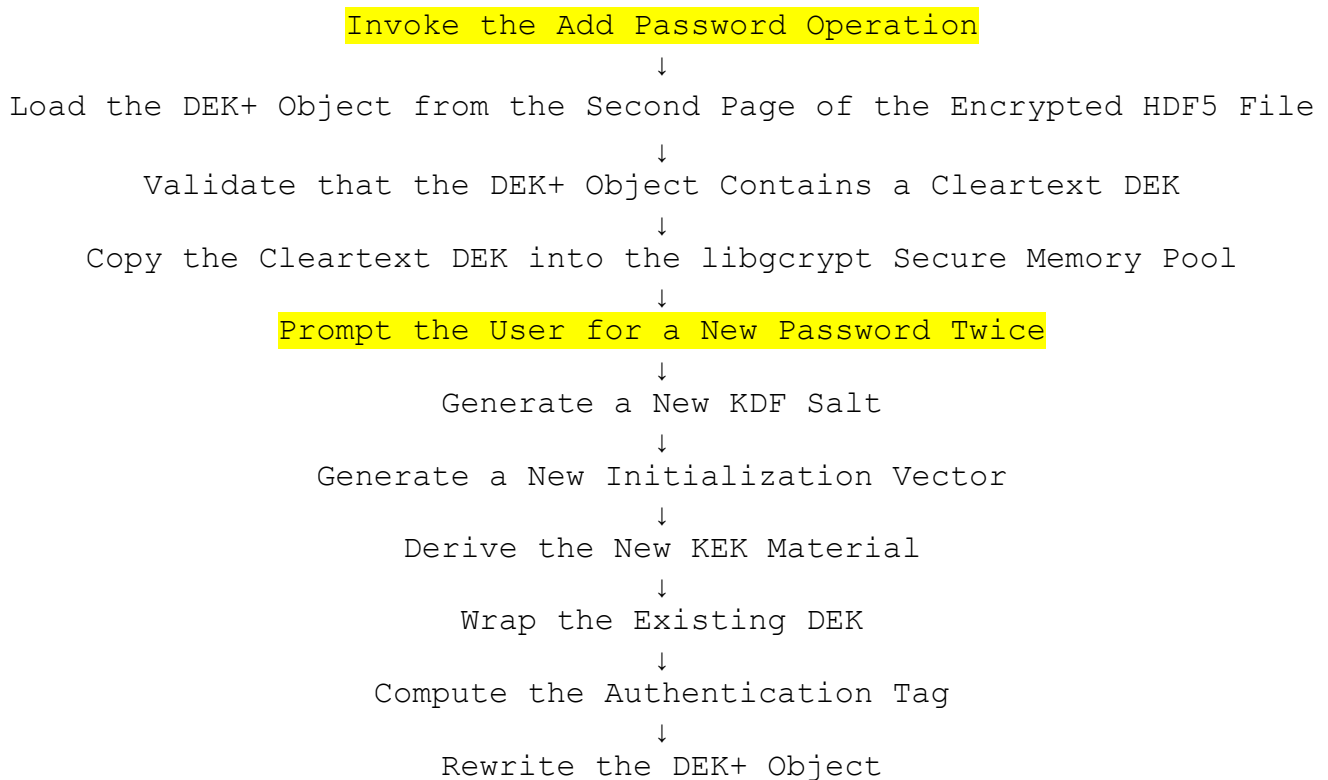


The workflow performs the same KEK rotation as the interactive process, except both the existing and replacement passwords are obtained through the public API rather than through a terminal prompt.

Since the underlying DEK remains unchanged, the encrypted HDF5 file data do not require re-encryption.

2.1.9 Add a password with Terminal-Based Input

- Adding a password is intended for DEK+ objects that are currently stored without password protection. Rather than changing the underlying DEK, this operation introduces password-derived KEK protection for an existing DEK.
- Password-management operations may be performed through either the h5keytools companion utility or the public Encryption VFD key management API. Both interfaces invoke the same implementation and therefore provide identical cryptographic behavior.

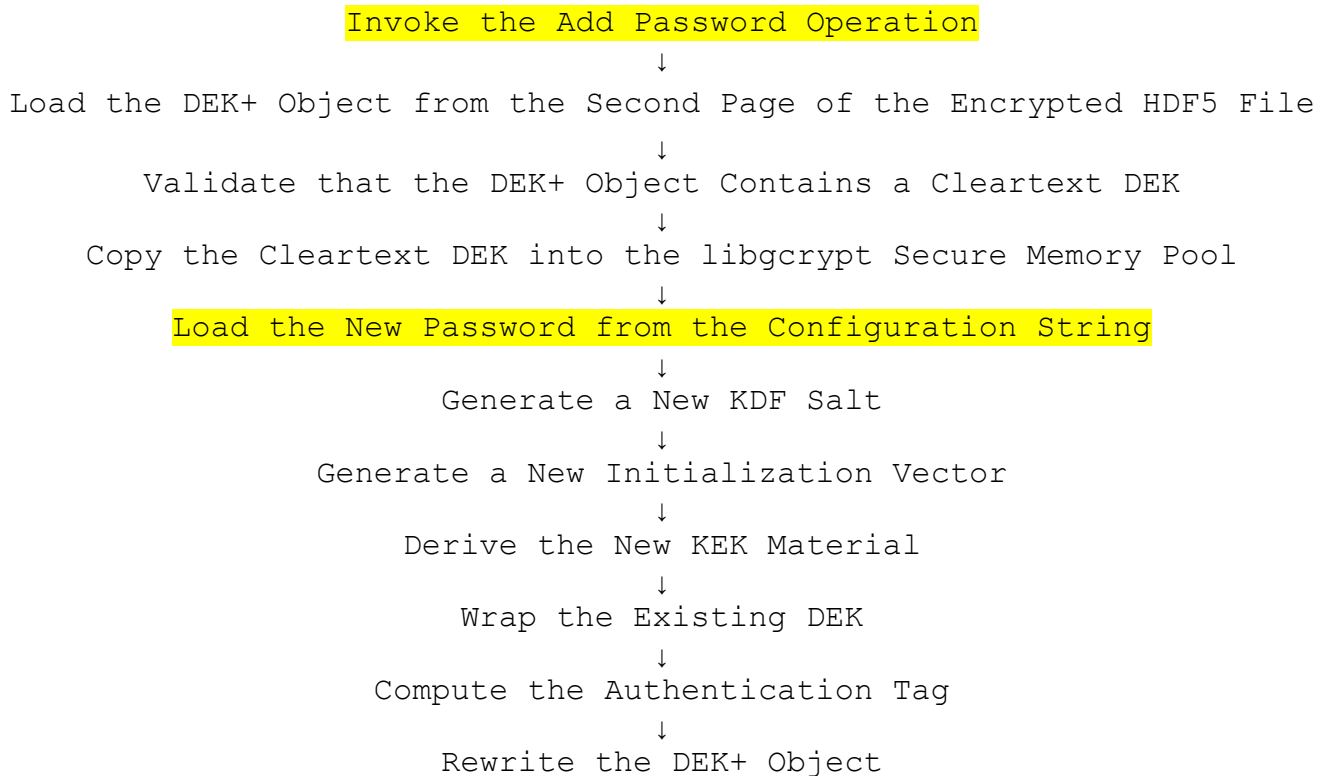


Adding a password restores password-based protection to an existing DEK+ object without altering the encrypted HDF5 data. Since only the KEK changes, the underlying DEK remains unchanged. The encrypted HDF5 file data do not require re-encryption.

2.1.10 Add a password without Terminal-Based Input

- This workflow performs the same password-addition operation described in Section 2.1.9, except the replacement password is provided through the public API or configuration interface rather than through terminal prompts.

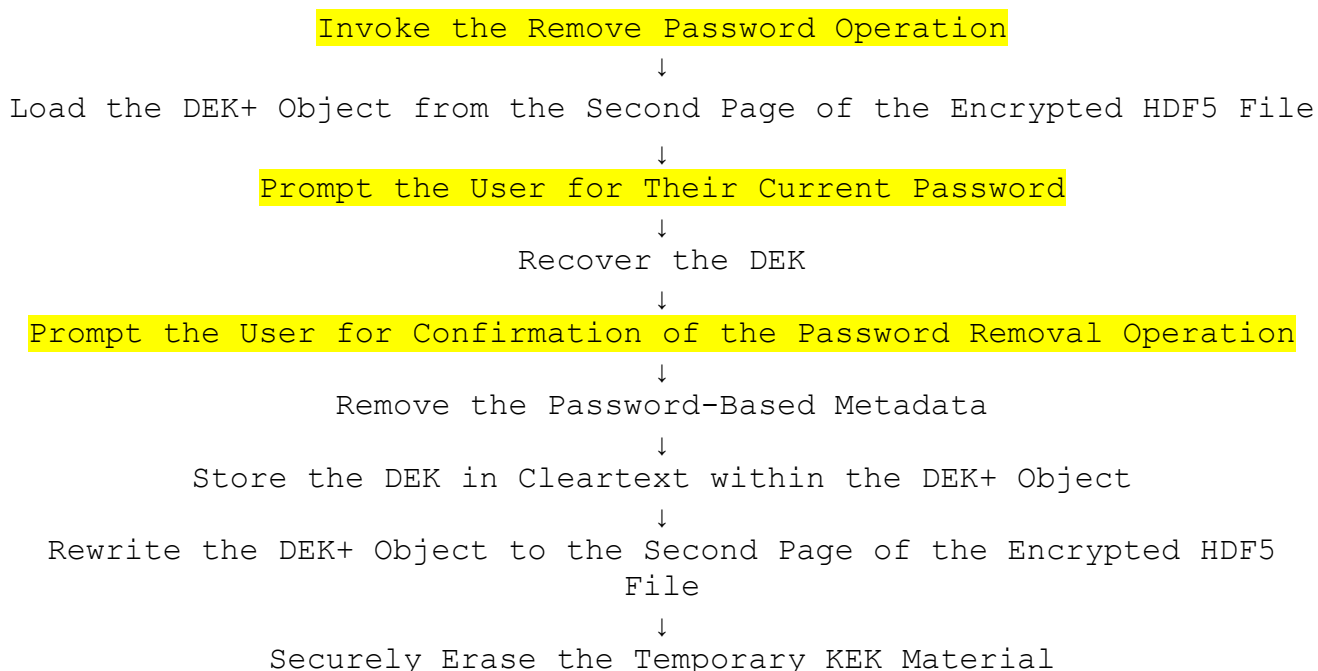
- Since only the KEK changes, the existing DEK and encrypted HDF5 data remain unchanged throughout the operation.
- The procedure below illustrates the non-interactive workflow. Highlighted components indicate where application-provided input is expected.



Adding a password restores password-based protection to an existing DEK+ object without altering the encrypted HDF5 data. Since only the KEK changes, the underlying DEK remains unchanged. The encrypted HDF5 file data do not require re-encryption.

2.1.11 Remove a password with Terminal-Based Input

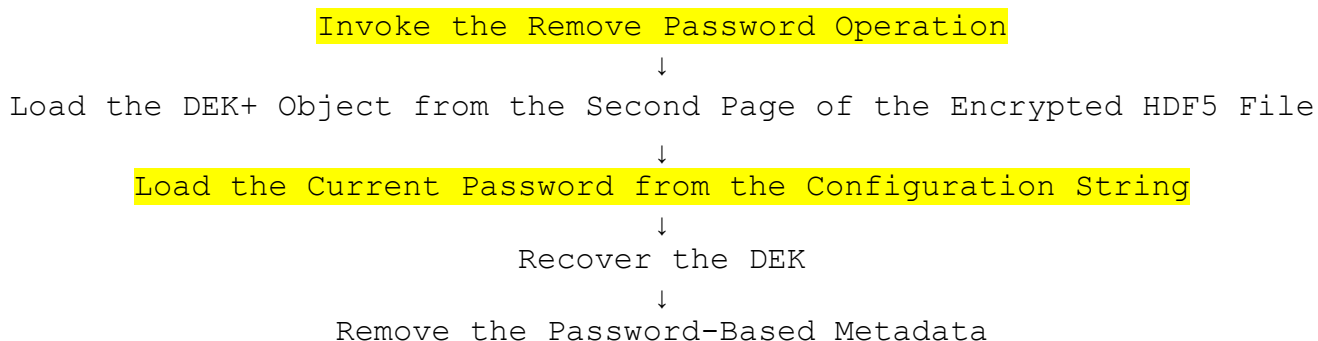
- Removing a password permanently removes password-based protection from the DEK+ object. Unlike a password change, this operation does not derive a replacement KEK. Instead, the recovered DEK is subsequently stored in cleartext within the DEK+ object.
- Because this operation substantially weakens the protection provided by the DEK+ format, it should require explicit user confirmation and should only be available when the appropriate security policy permits cleartext DEKs.
- The procedure below illustrates the interactive password-removal workflow. Highlighted components indicate where user interaction is expected.

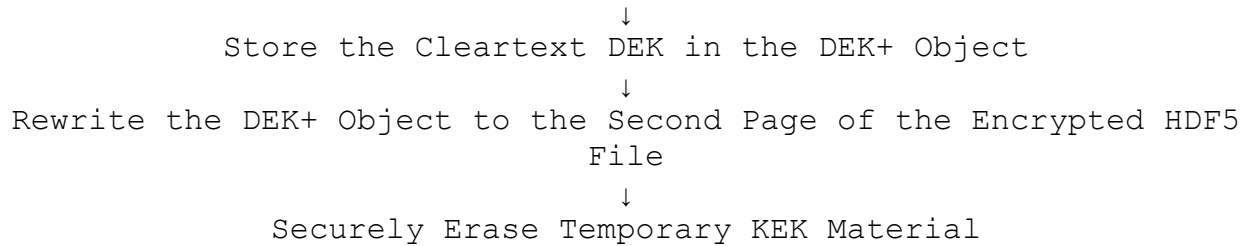


Removing the password intentionally changes the DEK+ security model by eliminating password-based protection entirely. Following successful authentication and user confirmation, the recovered DEK is written directly into the DEK+ object without deriving a replacement KEK. Consequently, this workflow should be reserved for trusted systems or specialized environments where cleartext DEK storage is explicitly permitted by policy.

2.1.12 Remove a password without Terminal-Based Input

- This workflow performs the same password-removal operation described in Section 2.1.11, except the current password is provided programmatically through the public API.
- Upon successful authentication, the recovered DEK is written to the DEK+ object in cleartext and all password-derived KEK material is removed from the object.
- The procedure below illustrates the non-interactive workflow. Highlighted components indicate where application-provided input is expected.

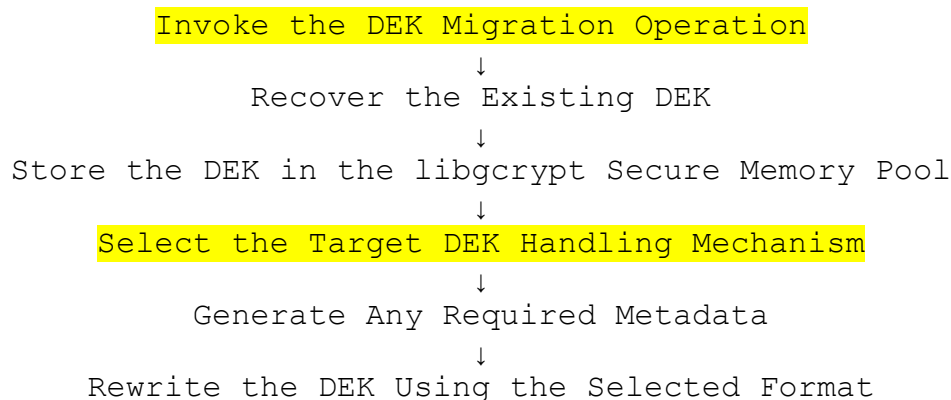


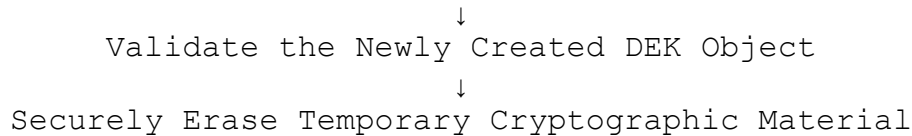


Removing the password intentionally changes the DEK+ security model by eliminating password-based protection entirely. Following successful authentication, the recovered DEK is written directly into the DEK+ object without deriving a replacement KEK. Consequently, this workflow should be reserved for trusted systems or specialized environments where cleartext DEK storage is explicitly permitted by policy.

2.1.13 Migrating Between Supported DEK Handling Mechanisms

- Although the initial implementation focuses on the DEK+ format, the overall Encryption VFD architecture is intentionally designed to support multiple DEK handling mechanisms. Future releases will therefore permit users to migrate between supported key storage mechanisms without requiring re-encryption of the associated HDF5 data.
- Regardless of the selected storage mechanism, the Encryption VFD ultimately consumes the same recovered 32-byte DEK. Consequently, migration consists only of recovering the existing DEK and rewriting that same DEK using a different supported representation. The encrypted HDF5 data remain unchanged throughout the operation.
- This functionality is not expected to be included in the initial implementation, but is anticipated to become available interactively through the `h5keytools` companion utility and programmatically through public API calls as additional DEK handling mechanisms are introduced.
- The procedure below illustrates the conceptual migration workflow. Highlighted components indicate where user or application interaction may occur.





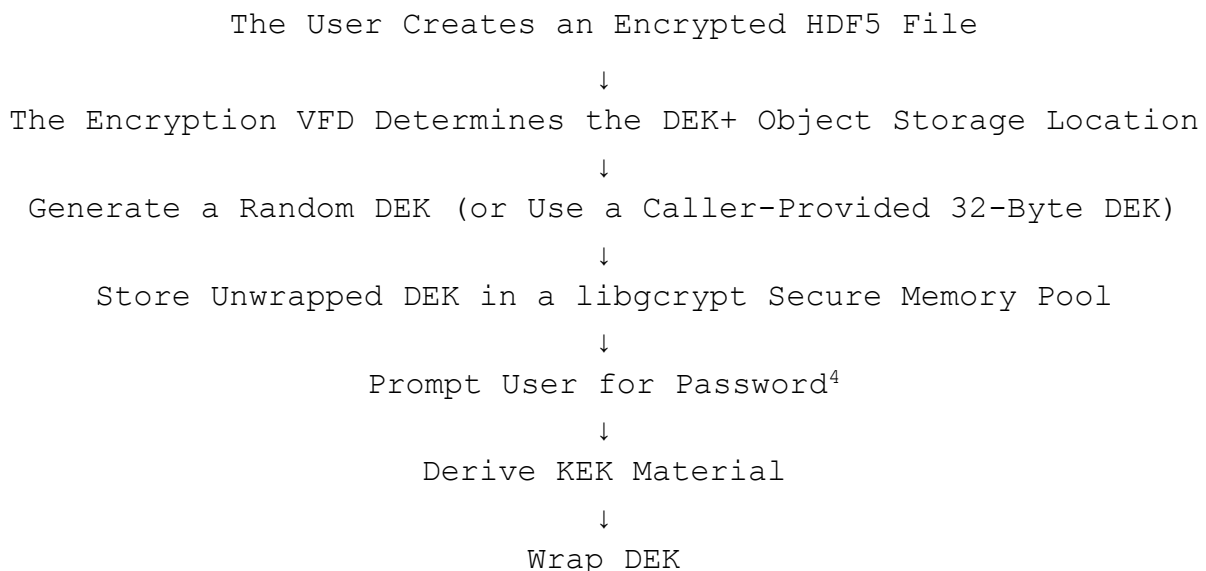
Because the Encryption VFD always operates on the recovered plaintext DEK rather than on the underlying storage representation, migration between supported DEK handling mechanisms does not require re-encryption of the protected HDF5 data. Instead, the existing DEK is recovered into the `libgcrypt` secure memory pool and immediately rewritten using the selected output mechanism. This capability is planned for a future release and is expected to be implemented through the `h5keytools` companion utility and public API calls using the same public key management interfaces described throughout this document.

3 Implementation Outline

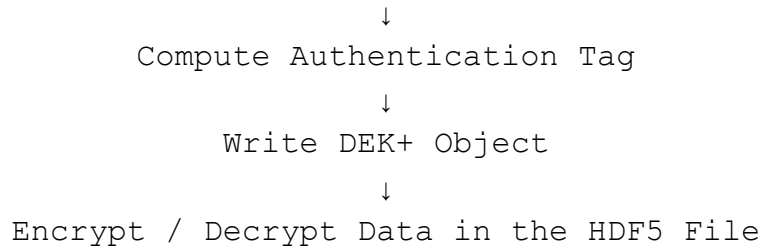
This section focuses on outlining the individual components of the local key generation solution.

3.1 High-Level Usage Description

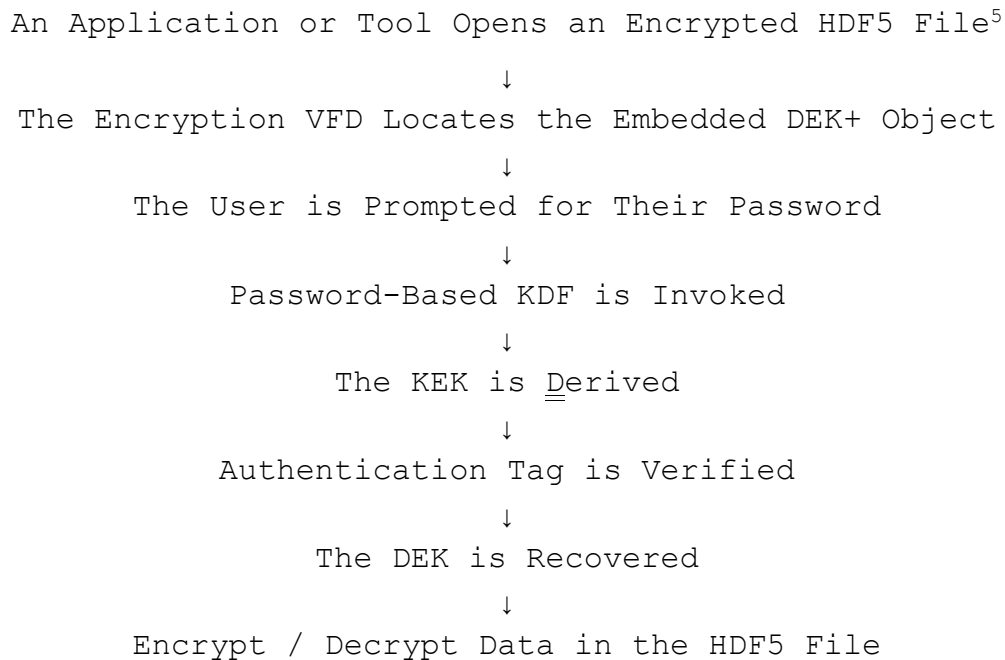
There are several major components necessary to facilitate the implementation and use of the local key generation solution. To illustrate each of these pieces, it is first important to establish what the typical workflow will look like when an HDF5 file is created using the Encryption VFD:



⁴ When permitted by the applicable build capabilities and runtime policy, the user may press Enter to select an empty-string password (""). Subsequent operations may bypass interactive password entry because the empty string is known in advance. The DEK remains wrapped and authenticated, but the representation provides no meaningful password-based access barrier and should be restricted to trusted automated or non-interactive environments.



Similarly, the typical workflow for opening an existing encrypted HDF5 file through the Encryption VFD is as follows:



In both situations, it is important to note that the KEK is derived in memory and only retained long enough to recover the DEK. In contrast, the decrypted DEK will be stored in the `libgcrypt` secure memory pool until the file is closed or the process is terminated. This secure retention removes the need to recover the DEK each time it is needed to process a page, while maintaining the at-rest protection of the serialized DEK. This also minimizes required user interaction. When a new data file is created or an existing data file is opened, the user is prompted for their password at most one time.

3.2 DEK+ Format Summary

To support the key generation solution, a simple binary DEK+ format will be adopted. A serialized DEK+ object consists of two principal components: a fixed-size header and a sequence of payload fields

⁵ Interaction may occur through an application, script, command-line utility, or another HDF5-compatible tool that uses the Encryption VFD.

whose lengths are recorded in the header. The header contains the DEK+ format version, serialized state flags, KDF parameters, cryptographic algorithm identifiers, payload lengths, and the other metadata required to interpret and validate the complete DEK+ object. These values are generated and processed by the implementation and are not intended to be modified manually by the user. Although the header includes explicit payload-length fields to support future format changes, the initial implementation requires fixed lengths for each supported representation. Taken as a whole, the serialized DEK+ object contains the information required to interpret and validate the object and, for a wrapped DEK representation, authenticate the object and recover the DEK. For a detailed description of the DEK+ format, see Section 5.2.

3.3 Password Usage

The local key-generation solution normally protects the DEK using password-derived KEK material. In interactive workflows, the user is prompted for a password when creating a new encrypted file or opening an existing file through the Encryption VFD. When combined with the KDF salt and parameters stored in the DEK+ object, the user-provided password allows the implementation to reconstruct the KEK material required to authenticate the object and wrap or recover the DEK. When terminal prompting is unavailable or undesirable, the password may instead be provided programmatically. Alternatively, where explicitly permitted by the installed build and runtime policy, `h5keytools` may convert a DEK+ object to a representation that does not require a non-empty password. Depending on the selected operation, this may produce either an empty-string password representation, which retains KEK-based wrapping and authentication, or a cleartext DEK representation, which removes KEK protection entirely. When the less-protected representation is no longer required, the user should restore non-empty-password protection through an Add Password or Change Password operation, as appropriate. The intended terminal consent and password-prompt behavior is described in section 3.3.1.

During interactive operations, the user will be prompted for their password through the terminal using an input routine that suppresses character echoing. This prevents the password from being displayed on the terminal and inadvertently captured in terminal logs or command histories. The entered password temporarily exists in cleartext and is copied into the `libgcrypt` secure memory pool. It is retained only long enough to derive the KEK material and is never written to disk or stored within the DEK+ object. Once the password has been collected, it is immediately provided to the configured Key Derivation Function (`scrypt` for the initial implementation) together with the KDF salt stored in the DEK+ object. The KDF produces 64 bytes of KEK material, with the first 32 bytes used as the AES-256-CBC wrapping key and the remaining 32 bytes used as the HMAC-SHA256 authentication key. After the key derivation process completes, the plaintext password buffer should be securely overwritten before being released, minimizing the amount of time that the password remains resident in application memory.

Since the derived KEK material constitutes secret cryptographic material, it should be allocated from the `libgcrypt` secure memory pool. Unlike the recovered DEK, however, the derived KEK exists only for the duration required to authenticate the DEK+ object and recover or rewrap the DEK. Once these operations are complete, the KEK material should be securely erased.

3.3.1 Terminal-Based Consent and Password Prompt

Interactive password-based key management operations require a terminal interface for displaying security information, collecting explicit user consent, and obtaining passwords without exposing secret input. The prompt design will use a layered, fail-safe approach in which a simple plain-terminal interface serves as the mandatory baseline and more sophisticated presentation mechanisms may be added in future implementations. The public prompt interface should remain portable and operating-system agnostic. Platform-specific terminal handling must be isolated behind internal helpers so that callers use the same prompt API on all supported systems. The initial implementation will focus on **Layer 1 — Plain Terminal Fallback**, including the platform-specific terminal handling necessary to suppress password echo. Layers 2 and 3, as separate enhanced capabilities, are reserved for future development.

3.3.1.1 Layer 1 — Plain Terminal Fallback

Role:

Mandatory baseline for interactive prompting and the universal fallback when richer terminal presentation is unavailable or inappropriate.

Behavior:

Layer 1 writes concise warning and explanatory text directly to `stderr`, requests explicit `y/N` consent, and then performs non-echoed password entry. When an operation creates or replaces a password, the user is prompted to enter the password a second time and the two inputs are compared before the operation proceeds.

Compatibility:

The presentation and prompt-control logic should rely on standard C library facilities wherever practical. Password echo suppression requires a small platform-specific backend beneath the otherwise portable prompt interface. The initial implementation is expected to support at least:

- * POSIX-compatible systems through `termios`;
- * Windows consoles through `GetConsoleMode()` and `SetConsoleMode()`.

The platform-specific code must remain isolated from the key-management, cryptographic, and policy layers.

Fallback Rule:

Layer 1 is used whenever terminal dimensions or capabilities cannot be determined, the terminal is too narrow for richer presentation, the terminal environment is limited, or a more advanced presentation layer is unavailable. Layer 1 must never fall back to visibly echoed password entry. If terminal echo cannot be disabled and no separately authorized non-interactive password source is available, the operation must fail safely. The Layer 1 workflow will perform the following general sequence:

1. Write a concise description of the requested operation and any applicable security warning to `stderr`.
2. Request explicit user consent using a `y/N` prompt.
3. Treat Enter, invalid input, or any response other than an explicitly accepted affirmative response as **No**.
4. Determine whether secure terminal password entry is available.
5. Save the current terminal-input state.
6. Disable terminal echo.
7. Read the password using bounded input handling.
8. Restore the previous terminal-input state.
9. When creating or replacing a password, repeat the non-echoed input sequence and compare the two entries.
10. Reject oversized or truncated input.
11. Return the collected password only after the complete prompt sequence succeeds.
12. Clear temporary password buffers after the password has been copied into the secure-memory allocation used for KDF processing.

Warnings and prompts will be written to `stderr` rather than `stdout`. This keeps interactive security messages separate from normal program output and reduces interference with output redirected by scripts or other tools. Password content must never be written to either output stream. The consent prompt must default to rejection. A representative Layer 1 interaction may take the following form:

*WARNING: This operation will create a password-protected DEK+ object.
The password will be required to recover the encryption key in the future.*

Continue? [y/N]:

Password:

Confirm password:

The password characters are not displayed while the user types. A newline may be written after each completed password entry so that subsequent output begins on a new terminal line. Security-reducing operations, such as removing password protection or exporting a cleartext DEK, must display operation-specific warnings before requesting consent. An affirmative response records interactive acknowledgement of the operation but does not replace the applicable build-capability, runtime-policy, DEK+ object-state, or operation-flag checks described in Section 3.4.

3.3.1.2 Layer 2 — Boxed In-Terminal View

Role:

Enhanced explanatory and consent presentation within the active terminal without opening a graphical window.

Behavior:

Layer 2 renders the warning and explanatory text inside an ASCII-framed panel before presenting the same explicit `y/N` consent prompt used by Layer 1. The password-entry behavior remains unchanged and continues to use the secure non-echoed input mechanism provided by the prompt backend.

Compatibility:

Layer 2 should depend only on terminal width and basic text-rendering capabilities. ASCII borders should be the default because they are broadly compatible. Unicode decoration may be supported later but must remain optional and must not be necessary for correct operation.

Fallback Rule:

If the terminal width cannot be determined reliably, the warning text may wrap incorrectly, the terminal reports limited capabilities, or rendering is otherwise uncertain, the implementation must immediately use Layer 1. Failure to render the enhanced panel must never prevent the plain fallback prompt from being used. Layer 2 is reserved for future development and is not required for the initial release.

3.3.1.3 Layer 3 — Extended Platform-Specific Secure Input Backend

Role:

Provide additional platform-specific hardening and terminal-state management while preserving the same public prompt interface.

Behavior:

Layer 3 expands the platform-specific secure-input implementation beyond the minimum functionality required by Layer 1. Potential additions include more comprehensive terminal-capability detection, improved state restoration, catchable-signal handling, cancellation behavior, retry controls, and tighter integration with platform-native console facilities.

Compatibility:

Platform-specific implementations are expected to include:

- POSIX terminal handling through `termios` and related signal or descriptor management;
- Windows console handling through `GetConsoleMode()`, `SetConsoleMode()`, and related console APIs.

All platform-specific differences must remain behind internal helper functions. The public prompting contract, password-policy behavior, and key-management operations must remain unchanged.

Fallback Rule:

If a Layer 3 capability is unavailable or fails before password collection begins, the implementation may fall back to the minimum secure-input behavior required by Layer 1. It must not fall back to visible password input. If the original terminal state cannot be obtained or echo cannot be disabled safely, the operation must fail unless an explicitly authorized non-interactive secret source has been supplied. Layer 3 hardening is reserved for future development. The initial release includes only the minimum platform-specific terminal handling required to provide secure non-echoed input in Layer 1.

3.3.1.4 Terminal Echo Suppression

Although the Layer 1 prompt interface is intended to remain operating-system agnostic, terminal echo suppression is a mandatory requirement for the initial release. The C standard library does not provide a portable API for modifying terminal echo behavior, so platform-specific implementation code is required beneath the portable interface.

On POSIX-compatible systems, the implementation should:

1. obtain the current terminal settings using `tcgetattr()`;
2. save the complete original settings;
3. clear the `ECHO` flag in a temporary copy;
4. apply the modified settings using `tcsetattr()`;

5. read the password;
6. restore the saved settings on every normal success and error path.

On Windows, the implementation should:

1. obtain the console input handle;
2. retrieve the current console mode using `GetConsoleMode()`;
3. save the complete original mode;
4. clear `ENABLE_ECHO_INPUT`;
5. apply the modified mode using `SetConsoleMode()`;
6. read the password;
7. restore the saved console mode on every normal success and error path.

The terminal state must be restored before returning from the prompt routine, including after input failure, oversized input, confirmation mismatch, policy rejection, or cancellation. Abrupt termination that cannot be intercepted by the process, such as `SIGKILL`, cannot provide a restoration guarantee.

3.3.1.5 Password Confirmation and Empty-String Passwords

Operations that establish a new password, including DEK+ object creation, Change Password, and Add Password, must request the password twice. If the two inputs differ, the implementation must:

- securely clear both temporary input buffers;
- report that the passwords do not match;
- leave persistent DEK+ content unchanged; and
- restart the password-entry sequence or return an error according to the selected retry policy.

KDF processing must not begin until password confirmation succeeds.

Pressing **Enter** without entering characters produces an empty-string password only when all applicable conditions are satisfied:

- empty-string password support is present in the current build;
- `allow_empty_password` is true;
- the selected operation permits creation or use of an empty-string password representation; and
- any required user acknowledgement has been obtained.

A zero-length password is distinct from a `NULL` password pointer. A zero-length password is a valid password input from which KEK material is derived. A `NULL` pointer indicates that no password was provided programmatically and may cause an interactive prompt to be requested.

3.3.1.6 Interactive and Non-Interactive Input

The prompt implementation must distinguish an interactive terminal from redirected or piped input. Redirected standard input must not automatically be treated as a secure password source. If the process does not have access to an appropriate interactive terminal, the operation must either:

- use a separately authorized password supplied through the public API or Encryption VFD configuration; or
- fail with an explicit error indicating that secure interactive input is unavailable.

This behavior is consistent with `LFBT_KEYMGMT_OP_BYPASS_PASSWORD_PROMPT`. That flag suppresses terminal collection only when all required password input has otherwise been supplied or when the selected DEK+ representation does not require password input. Environment variables should not be treated as a default password source because their contents may be exposed through process inspection, diagnostics, inherited environments, or debugging tools. Any future support for environment-based secrets must be explicitly enabled and documented as a separate non-interactive mechanism.

3.3.1.7 Initial Implementation Scope

The initial Layer 1 release will include:

- plain warning and explanatory text written to `stderr`;
- default-No `y/N` consent;
- operation-specific security warnings;
- bounded input handling;
- secure non-echoed password entry;
- password confirmation where required;
- empty-string password handling consistent with runtime policy;
- restoration of terminal settings on normal success and error paths;
- rejection of redirected or unsuitable interactive input;
- secure clearing of temporary password buffers; and
- a portable public prompt interface with isolated POSIX and Windows terminal backends.

The following capabilities are reserved for future development:

- Layer 2 ASCII-framed terminal presentation;
- automatic terminal-width and capability detection;
- optional Unicode presentation;
- expanded Layer 3 terminal hardening;
- catchable-signal-based terminal restoration;
- configurable retry limits and cancellation policies;
- protected file-descriptor secret sources;
- operating-system credential-store integration; and
- other non-interactive secret-provider mechanisms.

Future Layer 2 and Layer 3 enhancements must remain behind the same public prompt interface. Their addition must not change the key-management operations, runtime-policy fields, DEK+ object flags, password semantics, or distinction between interactive and non-interactive workflows described elsewhere in this document.

3.3.1.8 Failure and Cleanup Behavior

The prompt operation must fail without modifying persistent key material when:

- the user declines consent;
- the input source is not an appropriate interactive terminal;
- the original terminal state cannot be obtained;
- terminal echo cannot be disabled;
- terminal settings cannot be restored reliably on a normal execution path;
- input cannot be read;
- the password exceeds the supported input length;
- password confirmation does not succeed;
- an authorized non-interactive password was not supplied when interactive input is unavailable;
or
- the requested empty-string password or security-reducing behavior is not authorized.

Temporary ordinary-memory buffers used during input collection must be securely overwritten after the password has been copied into the libgcrypt secure memory pool or whenever the prompt operation fails. Platform-specific terminal resources and saved terminal state remain owned by the prompt backend and must be released or restored before the routine returns.

3.4 User-Interaction, Build Capability, and Policy Model

The implementation distinguishes between three independent control layers:

1. **Compiled capabilities** determine which security-sensitive implementation blocks are present in the resulting binary.
2. **Runtime policy and operation arguments** determine whether a present capability is authorized and requested for a particular invocation.
3. **Serialized DEK+ object flags** describe the persistent representation of an existing or newly written DEK+ object.

These layers serve different purposes and must not be treated as interchangeable. A preprocessor definition makes the associated implementation available. It does not request the functionality, authorize its use for a particular operation, or determine the state written into the DEK+ object.

For the initial implementation, all currently defined key management features will be compiled by default. Security-sensitive portions of the implementation will nevertheless be enclosed in explicit conditional-compilation blocks so that later builds may exclude them without restructuring the source code or requiring build-system-specific alterations. The relevant capability definitions currently include:

```
#define LFBT_KEYMGMT_ENABLE_EMPTY_PASSWORD
#define LFBT_KEYMGMT_ENABLE_CLEARTXT_DEK
#define LFBT_KEYMGMT_ENABLE_CLEARTXT_EXPORT
#define LFBT_KEYMGMT_ENABLE_EXTERNAL_DEKPLUS
```

Enabling all functionality in the default initial build changes the practical role of the build layer but does not eliminate it. Capability definitions should originate from a generated configuration header or a single `LFBT_keymgmt` configuration header. They should not be independently redefined throughout implementation files. In the initial implementation, the compiled-capability checks will normally succeed. They should nevertheless remain present so that:

- downstream or hardened builds may exclude selected capabilities;
- both present-capability and absent-capability paths can be exercised in test code;
- supporting secure build profiles within the implementation will not require a policy redesign; and
- operation flags and runtime policy cannot accidentally become substitutes for missing build checks.

A security-sensitive operation must satisfy all applicable conditions:

- the implementation block must be present in the current build;
- the selected public operation must permit the requested behavior;
- the runtime policy fields must authorize the behavior;
- explicit acknowledgement must be present when the operation reduces protection;
- the existing DEK+ object state must be compatible with the operation; and
- the resulting DEK+ object flags must accurately describe the representation written to storage.

3.4.1.1 Effect on Build Options

Build options should control whether source blocks are compiled. They should not be represented as DEK+ object flags and should not be passed as per-operation inputs. For the initial implementation, the build system should define all capability macros by default. A future hardened-build option may undefine one or more of these capabilities. For example, a future hardened build may omit the cleartext DEK storage and the cleartext export features from the default build while retaining password-protected and empty-password DEK+ support. When a capability is omitted, the public functions associated with that capability will remain linkable and return an explicit “capability not compiled” error. The compiled capability set will also be available programmatically through `LFBT_keymgmt_get_capabilities()`. Applications and `h5keytools` may use this function to determine whether a feature is present before presenting an operation to the user. Capability discovery does not authorize use of the reported feature; all applicable runtime policy, operation, acknowledgement, and serialized-state checks remain required.

3.4.1.2 Effect on Runtime Operation Flags

Operation flags continue to describe procedural behavior for one invocation. These flags do not indicate whether code was compiled and they do not describe the persistent DEK+ representation.

In the initial implementation, the following operation flags are planned to be implemented:

```
#define LFBT_KEYMGMT_OP_BYPASS_PASSWORD_PROMPT (1ULL << 0)
#define LFBT_KEYMGMT_OP_CONFIRM_SECURITY_REDUCTION (1ULL << 1)
#define LFBT_KEYMGMT_OP_ATOMIC_REPLACE (1ULL << 2)
#define LFBT_KEYMGMT_OP_RETURN_DEK (1ULL << 3)
#define LFBT_KEYMGMT_OP_EXTERNAL_COPY (1ULL << 4)
```

The exact naming convention remains subject to the initial implementation’s final naming convention and will be updated as needed. Each security-sensitive implementation block will include a prominent source-code warning documenting the following:

- the security property weakened by including the capability;
- the operations that can reach the capability;
- the runtime authorizations still required before it may be utilized; and
- the serialized DEK+ object state, if any, that the capability permits the implementation to read or produce.

For example, the source-code warning for enabling a cleartext DEK might take the following form:

```
#ifdef LFBT_KEYMGMT_ENABLE_CLEARTEXT_DEK
/*
 * SECURITY WARNING:
 *
 * This code permits a DEK+ object to contain an unwrapped, cleartext DEK.
 * Possession of such an object is sufficient to extract the DEK without
 * requiring password guessing or KEK derivation.
 *
 * Compiling this block does not authorize cleartext DEK operations. The
 * selected operation, runtime policy, explicit security-reduction
 * acknowledgement, and serialized object state must still be validated.
 */
...
#endif
```

The implementation must not infer build availability from an operation flag. For example, setting `LFBT_KEYMGMT_OP_CONFIRM_SECURITY_REDUCTION` records acknowledgement but does not compile, enable, or independently authorize cleartext-DEK storage. Because all capabilities are initially built by default, operation flags and the runtime policy become the principal barriers encountered during ordinary execution. This makes strict validation particularly important. Default compilation must not result in default authorization.

3.4.1.3 Effect on DEK+ Object Flags

DEK+ object flags continue to describe only the serialized state:

```
#define LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD (1u << 0)
#define LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK (1u << 1)
```

These flags must never be used as build controls or direct user requests.

A DEK+ object carrying `LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD` contains a wrapped and authenticated DEK whose KEK was derived from a zero-length password. Reading or producing this state requires:

- the empty-password implementation block to be compiled;

- runtime authorization through `allow_empty_password`; and
- an operation for which empty-password processing is valid.

A DEK+ object carrying `LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK` contains no KEK-protected DEK. Reading or producing this state requires:

- the cleartext-DEK implementation block to be compiled;
- runtime authorization through `allow_cleartext_dek`;
- a compatible operation; and
- explicit acknowledgement when the operation creates or preserves a less-protected representation.

These two flags are mutually exclusive. An empty password implies that KEK material exists. A cleartext DEK implies that no KEK (or KEK material) exists.

Users are not expected to construct the DEK+ operation-flag bitmasks or modify the DEK+ object header flags during ordinary use. Instead, the user selects a key management operation through the h5keytools companion utility or through the public key management API. The selected interface translates the requested behavior into the internal operation code, runtime policy values, and operation flags required by the shared DEK+ implementation. Build-time options dictate which security-sensitive capabilities are available in the resulting library and companion utility. Examples include support for empty-string passwords, cleartext DEK storage, cleartext DEK export, or the creation of external DEK+ copies (as a future capability). Enabling one or more build-time options makes the corresponding implementation available, but does not automatically request or authorize its use during normal execution. Further, each security-sensitive conditional-compilation block will contain a prominent source-code warning, and the build configuration documentation will identify the security implications of including the capability.

When the user invokes an operation, the selected front-end derives the appropriate runtime controls for the requested behavior. For example, an interactive password-change command permits terminal input, while a programmatic password-change request bypasses terminal prompting because the current and replacement passwords are provided through the public API. Similarly, a request to remove password protection causes the implementation to request the relevant security-reducing operation and, where applicable, collect explicit confirmation from the user.

The shared DEK+ implementation validates three independent conditions before performing a security-sensitive operation:

- The requested capability must be present in the current build.
- The requested operation and provided runtime policy must authorize the behavior.

- The existing DEK+ representation, user input, and requested output representation must be mutually consistent.

A runtime operation flag cannot enable functionality that was excluded at build time. If a caller requests an unavailable capability, the operation fails before the DEK+ object is modified or plaintext key material is exposed. The resulting error will identify the unsupported capability and, where appropriate, indicate which build-time option is required to successfully complete the desired operation. DEK+ object header flags are generated and interpreted internally. They describe the persistent representation of the DEK+ object, such as whether the object uses an empty-string password or stores a cleartext DEK. Users do not set these flags directly. After validating the requested operation, runtime policy, and build capabilities, the implementation derives and writes the appropriate header flags when creating or rewriting the DEK+ object.

3.5 Usage with the Encryption VFD

The local key management implementation will exist separately from both the Encryption VFD implementation and the core HDF5 library. For the provisional naming scheme used in this document, the shared implementation is referred to as `LFBT_keymgmt`. To maintain a clear separation of concerns, the `LFBT_keymgmt` components will be responsible for the following:

- DEK generation;
- password and runtime-policy validation;
- KEK derivation;
- DEK wrapping and recovery;
- authentication tag computation and verification;
- DEK+ header initialization and validation;
- serialization and deserialization; and
- orchestration of password management and migration operations.

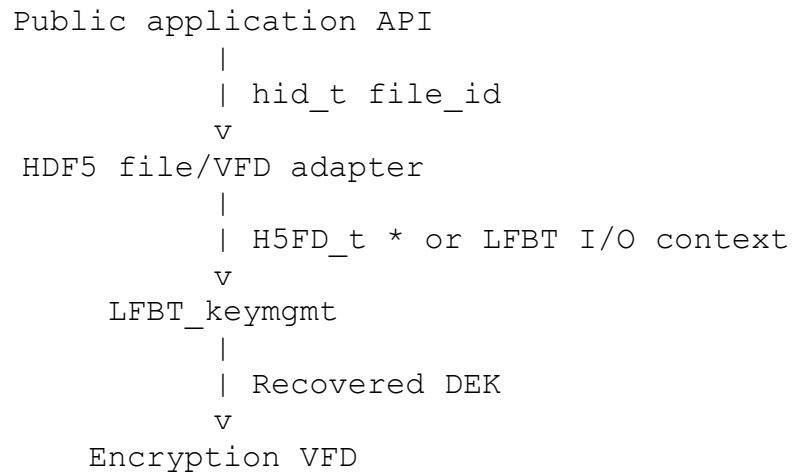
The Encryption VFD remains responsible for the following:

- initializing the process-level libgcrypt secure-memory facility, when initialization has not already been performed;
- allocating, retaining for the duration of an operation, and releasing temporary secret material through libgcrypt secure-memory allocation functions⁶;
- determining where the embedded DEK+ object resides (in the initial implementation, it is assumed to be the second page of the encrypted HDF5 file);
- providing read and write access to the DEK+ object's storage location;
- integrating key creation/recovery with file creation and open operations;
- retaining the recovered DEK for the lifetime of the associated HDF5 file; and

⁶ Ownership of a recovered DEK must be defined by the calling interface. When the Encryption VFD requests creation or opening a file, ownership should normally be transferred directly to the VFD-managed state rather than exposing the DEK to the application.

- consuming the recovered DEK for paged encryption and decryption operations.

When working with the non-interactive public API, the HDF5 library will receive the public file identifier, validate that it refers to an open encrypted HDF5 file, and route a key management request to the active Encryption VFD instance. The public interface should accept an HDF5 file identifier rather than exposing `H5FD_t` directly to an ordinary application. The intended layering is:



An internal HDF5 or Encryption VFD adapter may use the `H5FDctl()` interface to route operations to the target VFD instance. `H5FDctl()` should remain an internal mechanism that operates explicitly on `H5FD_t *` values; it should not be presented as the expected application-facing API.

3.6 H5keytools Companion Utility

To support and simplify migration between supported key management mechanisms, a companion utility named `h5keytools` is proposed. Its primary purpose is to create, inspect, re-protect, import, export, and migrate Encryption VFD key material while preserving the underlying DEK and avoiding re-encryption of the associated HDF5 data.

3.6.1 Purpose

The intent of `h5keytools` is to provide an independent interface for manipulating Encryption VFD key material regardless of the underlying storage mechanism. Rather than exposing multiple independent utilities for each supported format, the tool serves as a centralized mechanism for creating, converting, validating, and rekeying DEKs while preserving a consistent user interface and minimizing opportunities for incorrect usage.

The `h5keytools` utility is intended to invoke the same public DEK+ format interfaces implemented for the Encryption VFD rather than maintaining an independent implementation of the key management algorithms. This ensures that both components interpret the DEK+ format identically, minimizes duplicated cryptographic code, and guarantees that future changes to the DEK+ format are reflected consistently across all supported tools.

Although the Encryption VFD and `h5keytools` expose different user-facing workflows, both components should rely on the same underlying DEK+ implementation. Shared routines should cover DEK generation, DEK wrapping/unwrapping, HMAC computation and verification, header validation, serialization, deserialization, and secure-memory cleanup.

3.6.2 Supported Operations

The operations supported by the `h5keytools` utility are outlined in this section. For the initial release, the following functionality will be built out.

3.6.2.1 Create DEK

The `h5keytools` utility may be used to create a new 32-byte binary DEK for use with the Encryption VFD. This operation is intended for users who require a cleartext DEK for workflows outside of the DEK+ format, such as passing the DEK directly through an Encryption VFD configuration structure or configuration string, or subsequently importing it into a future external DEK+ object. Because this output is not password protected, the generated DEK should be handled as sensitive key material immediately after creation. If written to disk, the resulting file should be treated as equivalent to the unwrapped encryption key for all data protected by that DEK.

3.6.2.2 Create DEK+ Object

The `h5keytools` utility may create a new DEK+ object by generating or importing a 32-byte binary DEK, prompting the user for a password, deriving KEK material, wrapping the DEK, computing an authentication tag, and writing the resulting DEK+ object to the requested output location. For the initial implementation, this operation is expected to support storing the DEK+ object in a reserved page of an encrypted HDF5 file. Future versions may support storing complete DEK+ objects externally.

Similar DEK+ creation behavior may also occur inside the Encryption VFD when a new encrypted HDF5 file is created and the selected key management mode requires a DEK+ object. In that case, the Encryption VFD may generate the DEK, prompt for a password, create the DEK+ object, and retain the recovered DEK in a `libgcrypt`-based secure memory pool for immediate use. To avoid divergent behavior, the Encryption VFD and `h5keytools` should share the same DEK+ creation, wrapping, authentication, validation, and serialization routines.

3.6.2.3 Validate DEK+ Object

The `h5keytools` utility should provide a validation operation for DEK+ objects. At minimum, validations should confirm that the DEK+ header is well formed, the format version is supported, the expected lengths are present, algorithm identifiers are valid, and the total file size is consistent with the declared format. When a password is provided, validation may also derive the KEK material and verify the HMAC-SHA256 authentication tag. Structural validation confirms that the DEK+ object is well formed. Authentication validation confirms that the HMAC-SHA256 authentication tag is valid.

3.6.2.4 Change Password

Changing the password is fundamentally a KEK rotation operation. The DEK remains unchanged, while the password-derived KEK material, KDF salt, IV, wrapped DEK, and authentication tag are regenerated. The user provides the current password to recover the existing DEK, then provides a new password to rewrap that same DEK. This operation should be performed by `h5keytools`, not by the normal Encryption VFD open path. The Encryption VFD may unlock and consume the DEK, but password changes should be handled by the companion utility.

3.6.2.5 Remove Password

Removing a password converts a password-protected DEK+ object into a DEK+ object with no password protection. The DEK remains unchanged, but the DEK is unwrapped using KEK material derived from the provided password and then stored as cleartext. This operation should require explicit user confirmation and should only be permitted when saving a cleartext DEK is enabled. Removing the password substantially weakens the security of the DEK+ object and therefore should only be permitted when both:

- the implementation has been built with cleartext-DEK support enabled, and
- the current runtime policy explicitly authorizes password removal.

Interactive use should additionally require explicit user confirmation before the DEK+ object is rewritten without a password.

3.6.2.6 Add Password

Adding a password converts a DEK+ object that stores a cleartext DEK into a password-protected DEK+ object. The cleartext DEK is loaded from the DEK+ object and the DEK is wrapped using KEK material derived from the newly provided password (empty or otherwise) and a newly generated salt. This operation restores password-based protection to a cleartext DEK without requiring re-encryption of the associated HDF5 data.

3.6.2.7 Update KDF Parameters

The `h5keytools` utility may be used to update KDF parameters for an existing DEK+ object. This operation recovers the DEK using the existing password and KDF parameters, then rewraps the same DEK using newly selected KDF parameters. For example, a future implementation may increase the `scrypt` work factor to improve the resistance against offline password-guessing attacks. Updating KDF parameters associated with `scrypt` directly affects the work factor, and by extension, the cost of DEK recovery. Altering the work factor parameters does not alter the DEK or require re-encryption of the associated HDF5 data.

3.6.2.8 Future Format Migration

Future versions of the DEK+ format may introduce new header fields, additional authentication metadata, alternate KDFs, alternate wrapping algorithms, or support for multiple wrapped DEK records. The `h5keytools` utility should provide the primary migration path between supported DEK+ versions. A migration operation should recover the DEK using the existing format, then write the same DEK using the target format. Older implementations should reject unsupported versions rather than attempting to interpret unknown layouts.

3.6.3 Transferring Between Supported DEK Handling Mechanisms

For the initial implementation, the following DEK handling mechanisms are expected to be supported:

- A cleartext DEK provided directly through the Encryption VFD configuration structure or configuration string.
- A cleartext binary DEK stored in an external file with no additional content.
- The DEK+ format described in this document using a password-derived KEK.
- The DEK+ format using aa KEK derived from an empty-string password when explicitly permitted by the Encryption VFD configuration.
- The DEK+ format using no password-derived KEK (resulting in a cleartext DEK being stored) when explicitly permitted by the Encryption VFD configuration⁷.

Note: There has been internal discussion as to whether the DEK should be stored in one of the first pages of the encrypted HDF5 file. While this discussion is ongoing, it is important to note that cleartext DEKs and DEK+ objects may eventually be store either within an encrypted HDF5 file or in a separate external file. This addition would not directly change the remainder of the discussion, but would introduce additional supported key storage mechanisms and will need to be supported by the proposed tool.

Because all supported mechanisms ultimately provide the same 32-byte binary DEK to the Encryption VFD, converting between formats requires only recovering the DEK into the `libgcrypt` secure memory pool and rewriting it using the desired output format. At no point is the associated HDF5 data modified, as the DEK itself remains unchanged throughout the conversion process. Fundamentally, three conversions should be considered as described below.

3.6.3.1 Convert from Secure to Secure

Converting between secure storage mechanisms (for example, changing the KEK password or migrating between different DEK+ object instances) should be considered a standard operation. During these operations, the DEK is recovered into the `libgcrypt` secure memory pool, re-protected using the

⁷ In this representation, the DEK is not wrapped and is stored as unsecured cleartext.

requested output format, and immediately removed from any temporary buffers once the conversion completes.

3.6.3.2 Convert from Insecure to Secure

When converting from a cleartext DEK provided by the user to the DEK+ format, `h5keytools` will assume that the provided DEK is already cryptographically suitable for use with AES-256 (at least for the initial release). Since the utility cannot determine how the key was originally generated, it is the responsibility of the user to ensure that the provided DEK was generated using a cryptographically secure random number generator.⁸ Imposing a predictable, reused, or otherwise weak DEK into the DEK+ format protects the key only while it is stored at rest; it cannot improve the entropy or security of the underlying DEK itself.

3.6.3.3 Convert from Secure to Insecure

Exporting a DEK from a protected format into a cleartext representation fundamentally changes the security model of the Encryption VFD. Although the recovered DEK exists only within the `libgcrypt` secure memory pool during normal operation, exporting the DEK necessarily copies it into ordinary application memory before writing it to persistent storage. Once written as a cleartext binary file or embedded directly within an Encryption VFD configuration, the DEK is no longer protected by the password-derived KEK, the authenticated DEK+ format, or the `libgcrypt` secure memory pool subsystem.

From this point forward, any process, user, backup system, or adversary capable of reading the exported DEK obtains the complete encryption key required to decrypt all data protected by the DEK. Unlike attacks against the DEK+ format, which require successful password recovery through the KDF, attacks against a cleartext DEK require only access to the exported key itself. Exporting the DEK permanently bypasses every protection provided by the DEK+ format. Once exported, possession of the cleartext DEK alone is sufficient to decrypt all data protected by the key. Password recovery, KEK derivation, authentication-tag verification, and all other DEK+ security mechanisms are no longer involved.

3.6.4 Build-Time Protection

For the initial implementation, support for cleartext DEK export will be compiled by default but will be enclosed within the `LFBT_KEYMGMT_ENABLE_CLEARTEXT_EXPORT` conditional-compilation block. Compilation makes the capability available for use but does not provide authorization for use. Exportation will require runtime policy authorization and explicit acknowledgement of the security

⁸ A user who does not wish to assume responsibility for generating a cryptographically suitable DEK should use the password-based key management mechanism described in this document and allow `libgcrypt` to generate the DEK.

reduction. A future hardened build may omit the capability by leaving `LFBT_KEYMGMT_ENABLE_CLEARTEXT_EXPORT` undefined.

As an additional safety measure, `h5keytools` should refuse to overwrite an existing output key by default. Since the utility operates directly on encryption keys, unintentionally replacing an existing key may permanently prevent recovery of the associated HDF5 data. Consequently, overwriting an existing output file should require an explicit command-line option (for example, `--force` or `--overwrite`) and should generate an additional warning before proceeding. This behavior provides consistency with the overall philosophy of the proposed implementation, which favors secure defaults while requiring explicit user acknowledgement before performing potentially destructive operations.

4 Theory of Operations

To avoid conflicting with other Encryption VFD documentation, this section will focus on the general operations that will interact with the local key generation solution. Further interactions may be added as necessary.

4.1 Create a New DEK+ Object

1. The Encryption VFD determines that a new DEK+ object must be created.
2. The user is prompted for a password, including an empty-string password when the application build capabilities, runtime authorizations, and operation arguments allow one.
3. A random 32-byte DEK is generated and stored in the `libgcrypt` secure memory pool (unless the caller supplies an authorized existing 32-byte DEK and sets the applicable operation argument).
4. A random salt and IV are generated.
5. The KEK material is derived using `scrypt`.
6. The DEK is wrapped using `AES-256-CBC`.
7. An `HMAC-SHA256` authentication tag is computed.
8. The DEK+ object is written to file automatically (either to an external file or within the second page of an HDF5 file created through the Encryption VFD).
9. Temporary KEK material is securely erased.
10. The DEK remains resident in the `libgcrypt` secure memory pool for use by the Encryption VFD until the HDF5 file is closed or until an unrecoverable error causes the Encryption VFD to shut down and release the `libgcrypt` secure memory pool.

4.2 Interacting with an Existing Encrypted File

1. The Encryption VFD determines that an encrypted key already exists for the target HDF5 file.

2. The DEK+ object header is read and validated to ensure that the file format and cryptographic parameters are supported.
3. If required by the DEK+ object flags and Encryption VFD configuration, the user is prompted for their password through the terminal. Empty-string password operation is permitted only when both the DEK+ object metadata and the current Encryption VFD configuration explicitly allow it.
4. The KEK material is reconstructed using the stored KDF parameters, KDF salt, and user-provided password, including an empty-string password when explicitly permitted. If the DEK+ object carries `LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK`, password-based KDF processing, KEK derivation, wrapped DEK authentication, and unwrapping are not performed. Under the current format, this representation therefore provides neither confidentiality nor keyed integrity protection for the stored DEK. Structural validation and runtime-policy validation must still be performed before the DEK is accepted.
5. For a wrapped DEK representation, the `HMAC-SHA256` authentication tag is recomputed and verified before any decryption operation is performed.
6. Upon successful authentication of a wrapped DEK representation, the wrapped DEK is decrypted using `AES-256-CBC` and stored in the `libgcrypt` secure memory pool. For a cleartext representation, the validated 32-byte DEK payload is copied directly into secure memory.
7. Temporary KEK material and other intermediate cryptographic buffers are securely erased from memory.
8. The recovered DEK remains resident in the `libgcrypt` secure memory pool for the lifetime of the HDF5 file and is used by the Encryption VFD for subsequent encryption and decryption operations.

4.3 Rotating the KEK using `h5keytools`

1. The user invokes `h5keytools` with a rekey operation.
2. The existing DEK+ object is opened and validated, and the user is prompted for their current password through the terminal.
3. The existing KEK material is reconstructed, the authentication tag is verified, and the wrapped DEK is successfully recovered into the `libgcrypt` secure memory pool.
4. The user is prompted for a new password. If an empty password is requested, the operation proceeds only when `allow_empty_password` is enabled.
5. A new random KDF salt and initialization vector are generated.
6. A 64-byte block of KEK material is derived using the updated password and newly generated KDF parameters.
7. The existing DEK is wrapped using the newly derived KEK, and a new `HMAC-SHA256` authentication tag is computed.

8. The DEK+ object is atomically written using the updated header, salt, IV, wrapped DEK, and authentication tag.
9. Temporary KEK material and the recovered DEK are securely released after the operation completes, unless ownership of the DEK has been explicitly transferred to an active Encryption VFD instance.

5 Technical Details

This section expands upon the implementation outline presented earlier in this document by describing the proposed DEK+ format, public interfaces, internal helper functions, and execution flow necessary to integrate password-based key management into the Encryption VFD. The intent is not to provide a complete implementation specification, but rather to define the architectural components, interfaces, and cryptographic responsibilities required to produce a consistent implementation. Where appropriate, the functionality described throughout this section is expected to be shared between the Encryption VFD and the `h5keytools` companion utility to minimize duplicated logic and ensure consistent interpretation of the DEK+ format.

5.1 Encryption VFD Overview

As of the time of writing, the Encryption VFD allocates a secure memory pool using `libgcrypt` during the initialization phase. In short, a `libgcrypt` secure memory pool is a protected memory region that `libgcrypt` attempts to lock in physical memory so that it is not ordinarily paged to disk. It also includes allocation and access protections intended to reduce certain memory-disclosure and memory-corruption risks. Accordingly, unwrapped keys are stored in this memory pool. Additionally, using the version of the Encryption VFD available at commit tag `14ca76b8072351f0bd4270a29c5958489702763c`, two distinct methods are currently supported for passing in a cleartext key:

1. The user provides an unwrapped DEK directly into the Encryption VFD through a File Access Property List (FAPL):
 - i. The DEK may be provided as part of the `LFBT_crypt_vfd_config_t` structure
 - ii. The hex-encoded DEK can be set using the Encryption VFD configuration string
2. The DEK is stored in a separate file as a binary blob with no additional contents. To access this DEK, the file path is provided through one of the following methods:
 - i. As part of an instance of the `LFBT_crypt_config_t` structure
 - ii. As part of the Encryption VFD configuration string
 - iii. Using the `LFBT_CRYPT_KEY_PATH` environment variable

As of the time of writing, a user may freely switch among the supported methods for providing the DEK. The introduction of the DEK+ format will retain this freedom, with the understanding that the user may need to recover the cleartext version of the wrapped DEK from the DEK+ format should they wish to provide the DEK through a different supported mechanism. Independent of the key distribution strategy, the Encryption VFD will be responsible for maintaining the mechanisms used to load a DEK.

5.2 DEK+ Format

The DEK+ format is an application-defined binary format used to store the information necessary for recovering a 32-byte binary DEK. For the initial implementation, the DEK+ object will be stored within a reserved page of an encrypted HDF5 file created through the Encryption VFD. Support for storing a complete DEK+ object externally is reserved for future development. A password-protected DEK+ object does not store the cleartext DEK, user password, or derived KEK material. Instead, it stores the metadata and encrypted key material required to reconstruct the KEK from the user password, authenticate the DEK+ contents, and unwrap the DEK. An explicitly authorized cleartext DEK representation stores the DEK directly and omits password-derived KEK protection. Common general-purpose private-key containers such as PKCS#8, PKCS#12, and PEM are not used because this design protects a symmetric DEK rather than an asymmetric private key or certificate chain. A compact application-defined binary format is sufficient for the initial implementation and allows the Encryption VFD and `h5keytools` to share a single implementation of the DEK+ format serialization, validation, wrapping, and recovery.

5.2.1 DEK+ Object Layout

Version 1 of the DEK+ format consists of a fixed-size header followed by fixed-size payload fields. Although the payload fields are described as variable-length in the header for future compatibility, the initial implementation utilizes exact field sizes.

Layout: DEK+ Format

byte	byte	byte	byte
Magic – fixed file identifier			
Three bytes Reserved for Future Use			Version
Flags			
KDF ID			
KDF N			
KDF r			
KDF p			

KDF output length			
Cipher ID			
Cipher mode			
MAC ID			
DEK length			
Salt length			
IV length			
DEK payload length			
MAC length			
KDF salt (16 bytes in the initial implementation)			
...			
IV (16 bytes in the initial implementation)			
...			
Wrapped DEK (48 bytes in the initial implementation)			
...			
Authentication tag (32 bytes in the initial implementation)			
...			

[illegible]

Fields: Content of the DEK+ Format

Field	Size	Description
Magic	8 bytes	Fixed file identifier, expected to be EVFDKEY\0.
Reserved	3 bytes	Reserved for future use. Must be written as zero and ignored on read unless defined by a future version.
Version	1 byte	DEK+ format version. Version 1 indicates the initial implementation.
Flags	4 bytes	Bitmask describing the DEK+ policy and optional behavior.
KDF ID	4 bytes	Identifier for the password-based KDF. Version 1 uses <code>script</code> .
KDF N	4 bytes	<code>script</code> CPU/memory cost parameter. Version 1 default is 32768.
KDF r	4 bytes	<code>script</code> block-size parameter. Version 1 default is 8.

KDF p	4 bytes	<code>script</code> parallelization parameter. Version 1 default is 1.
KDF output length	4 bytes	Number of bytes produced by the KDF. Version 1 default is 64.
Cipher ID	4 bytes	Identifier for the cipher used to wrap the DEK. Version 1 uses AES-256.
Cipher mode	4 bytes	Identifier for the cipher mode. Version 1 uses CBC.
MAC ID	4 bytes	Identifier for the authentication algorithm. Version 1 uses HMAC-SHA256.
DEK length	4 bytes	Length of the unwrapped or cleartext DEK. Version 1 requires 32.
Salt length	4 bytes	Length of the KDF salt. Version 1 requires 16.
IV length	4 bytes	Length of the CBC IV. Version 1 requires 16.
DEK payload length	4 bytes	Length of the serialized DEK payload. Version 1 requires 48 bytes for a wrapped DEK and 32 bytes for a cleartext DEK.
MAC length	4 bytes	Length of the authentication tag. Version 1 requires 32 bytes for a wrapped DEK representation and 0 for a cleartext DEK representation.
KDF salt	16 bytes	Random salt used with the password to derive the KEK material.
IV	16 bytes	Random CBC initialization vector used to wrap the DEK.
DEK Payload	48 bytes or 32 bytes	For a wrapped DEK representation, this field contains the 48-byte AES-256-CBC ciphertext, including padding. For a cleartext DEK representation, it contains the 32-byte unwrapped DEK.
Authentication tag	32 bytes	HMAC-SHA256 output computed over the header, salt, IV, and wrapped DEK. This field is absent when <code>LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK</code> is set.

The DEK+ format containing a wrapped DEK in the initial implementation is expected to be 180 bytes. A cleartext DEK representation is expected to be 100 bytes: the 68-byte header followed by a 32-byte cleartext DEK payload. The total serialized length is calculated from the validated header length fields and is not universally fixed at 180 bytes. When a wrapped DEK is utilized, the 180 bytes are sourced as follows:

Component:	Size
Header	68 bytes
KDF salt	16 bytes
IV	16 bytes
Wrapped DEK	48 bytes
HMAC-SHA256	32 bytes

Total

180 bytes

5.2.2 DEK+ Header Structure

```

/*****
 *
 * Structure: LFBT_dekplus_hdr_t
 *
 * Purpose:
 *
 *   Defines the in-memory representation of the fixed-size header associated
 *   with a DEK+ object. The fields in this structure describe the DEK+ format
 *   version, serialized object state, password/KDF parameters, wrapping cipher
 *   parameters, authentication parameters, and the lengths of the payload fields
 *   that follow the fixed header.
 *
 *   A DEK+ object may be embedded within the second page of an encrypted HDF5
 *   file. Support for storing this object in an external file is reserved for
 *   future development. Consequently, this structure describes the DEK+ format
 *   itself and does not imply a particular storage location.
 *
 *   This structure does not contain the variable-length DEK+ payload. In the
 *   normal password-protected initial implementation representation, the
 *   serialized header is followed by the KDF salt, CBC initialization vector,
 *   wrapped DEK ciphertext, and authentication tag. Alternate representations,
 *   including the explicitly authorized cleartext DEK representation, are
 *   interpreted according to the flags and field lengths stored in this header.
 *
 *   This structure must not contain secret key material. The user password,
 *   derived KEK material, plaintext or cleartext DEK, and temporary decrypted
 *   buffers must not be stored in this structure.
 *
 * Serialization:
 *
 *   This structure is an in-memory representation and must not be written
 *   directly to persistent storage. Multi-byte integer fields are serialized
 *   explicitly in little-endian byte order. Byte-array fields are serialized as
 *   exact byte sequences. Implementations must not depend on compiler padding,
 *   native structure layout, or host byte order.
 *
 * Fields:
 *
 *   uint8_t magic[8]: Fixed DEK+ object identifier used to distinguish the DEK+
 *   format from unrelated data and to reject obviously invalid input. The
 *   expected initial implementation value is "EVFDKEY\0".
 *
 *   uint8_t reserved[3]: Reserved bytes for future use. For the initial
 *   implementation, these bytes should be set to zero and ignored on read
 *   unless defined by a future format version. They are included to
 *   preserve 32-bit alignment for the remaining header fields.
 *
 *   uint8_t version: DEK+ format version. Included to allow for future
 *   versions to add fields, change algorithms, or support additional
 *   wrapped-key records while allowing older implementations to reject

```

```

*      unsupported formats safely. A value of 1 indicates the initial
*      implementation.
*
*
*      uint32_t flags: Bitmask describing the serialized state and interpretation
*      of the DEK+ object. A value of zero indicates the normal, non-empty
*      password representation. In the initial implementation, the values
*      include LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD and
*      LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK. These flags are derived by the
*      implementation from an authorized operation and are not intended to be
*      set directly by users. See section 5.2.3 for complete header flag
*      definitions, mutual-exclusion rules, and validation requirements.
*
*      uint32_t kdf_id: Identifier for the password-based key derivation function
*      used to derive KEK material from the user password and stored salt.
*      The initial KDF selected is scrypt, making the default value of this
*      field GCRY_KDF_SCRYPT. This algorithm was selected to make the
*      password-derived KEK recovery more resistant to brute-force attacks
*      than a simple hash or a lower-cost KDF. This field is zero when the
*      cleartext-DEK representation does not use a KDF.
*
*      uint32_t kdf_n: The scrypt CPU/memory parameter (N in the documentation).
*      To satisfy the libgcrypt requirements, N must be greater than 1 and be
*      a power of two. It is suggested to use a value of at least 2^14 for
*      interactive logins. As such, a value of N = 32768 will be used as the
*      default for the initial implementation. This field is zero when the
*      cleartext-DEK representation does not use a KDF.
*
*      uint32_t kdf_r: The scrypt block size parameter (r in the documentation). A
*      default size of 8 will be used for the initial implementation. This
*      field is zero when the cleartext-DEK representation does not use a
*      KDF.
*
*      uint32_t kdf_p: The scrypt parallelization parameter (p in the
*      documentation). To maintain simple and predictable derivation
*      behavior, a default value of 1 will be used for the initial
*      implementation.
*
*      uint32_t kdf_out_len: The desired length of the derived key, in bytes,
*      produced by the KDF (len in the documentation). For the initial
*      implementation, a default value of 64 will be used. The first 32 bytes
*      of the derived material will be used as the AES-256-CBC wrapping key.
*      The second 32 bytes are used as the HMAC-SHA256 authentication key
*      (used for validation). This field is zero when the cleartext-DEK
*      representation does not use a KDF.
*
*      uint32_t cipher_id: Identifier for the cipher used to wrap the DEK. For the
*      initial release, GCRY_CIPHER_AES256 will be used as the default
*      cipher. Although only one cryptographic configuration is supported in
*      the initial implementation, algorithm identifiers are included to
*      support future migration and explicit format-version validation
*      without redesigning the file format. This field is zero when the
*      cleartext-DEK representation does not use a wrapping cipher.
*
*      uint32_t cipher_mode: Identifier for the cipher mode used to wrap the DEK.
*      Necessary for utilizing algorithms with configurable modes. Since
*      AES-256 will be used to secure the DEK in the initial implementation,

```

```

*      AES-256-CBC will be used as the default value in the initial
*      implementation. Of note, CBC does not provide authentication; the
*      wrapped DEK and associated DEK+ object content must be authenticated
*      separately using an encrypt-then-MAC construction. This field is zero
*      when the DEK is stored in cleartext.
*
*      uint32_t mac_id: Identifier for the message authentication algorithm. When
*      using an encryption algorithm and/or mode that does not provide
*      authentication, an authentication algorithm must be specified. For the
*      initial implementation, HMAC-SHA256 will be used as the default
*      configuration. This selection provides integrity and tamper detection
*      for the wrapped DEK and selected DEK+ metadata and is necessary
*      when using AES-256-CBC. This field is zero when no authentication
*      algorithm is used.
*
*      uint32_t dek_len: The length of the DEK represented by the DEK+ object.
*      For the initial implementation, the Encryption VFD requires a 32-byte
*      AES-256 key. As such, the default unwrapped DEK length is 32 bytes.
*
*      uint32_t salt_len: The length of the KDF salt, in bytes, stored after the
*      header. Salt is a non-secret value that must be stored with the key
*      file and, when used with the user-provided password, is used to derive
*      the KEK. For the initial implementation, a default salt length of 16
*      bytes will be used. This field is zero when no KDF salt is stored.
*
*      uint32_t iv_len: The length of the CBC initialization vector (IV), in bytes,
*      stored after the salt. When AES-256 is used in CBC mode, a fresh IV is
*      required for each DEK wrapping operation. Since AES has a 16-byte
*      block size, the default value for the IV length will be 16. This field
*      is zero when the DEK is stored as cleartext.
*
*      uint32_t dek_payload_len: The length, in bytes, of the DEK payload. For the
*      initial implementation, the wrapped DEK representation requires 48
*      bytes of AES-256 ciphertext, including full-block padding. For a
*      cleartext DEK representation, this field is expected to be 32 bytes.
*
*      uint32_t mac_len: The length of the authentication tag, in bytes, stored
*      after the wrapped DEK ciphertext. This is the full HMAC-SHA256 output
*      used to authenticate the wrapped DEK and selected DEK+ metadata. This
*      field is zero when no password-derived authentication tag is stored.
*
*****/

```

```

typedef struct LFBT_dekplus_hdr_t {
    uint8_t magic[8];

    uint8_t reserved[3];
    uint8_t version;
    uint32_t flags;

    uint32_t kdf_id;
    uint32_t kdf_n;
    uint32_t kdf_r;
    uint32_t kdf_p;
    uint32_t kdf_out_len;

    uint32_t cipher_id;

```

```

uint32_t cipher_mode;
uint32_t mac_id;

uint32_t dek_len;
uint32_t salt_len;
uint32_t iv_len;
uint32_t dek_payload_len;
uint32_t mac_len;
} LFBT_dekplus_hdr_t;

```

The `version` field is intentionally stored as a single byte. To preserve alignment, the three prior bytes are reserved for future use so that the remainder of the header remains aligned on 32-bit boundaries. For the initial implementation, the three reserved bytes should be written as zero.

5.2.3 DEK+ Header Flags

The DEK+ header flags describe the serialized state and interpretation of a DEK+ object. They are not intended to serve as direct user configuration settings. Users and applications instead select key management operations and runtime policies through the Encryption VFD configuration, the public key management API, or the h5keytools companion utility. When a permitted operation creates or rewrites a DEK+ object, the shared implementation derives and stores the appropriate header flags. When opening an existing DEK+ object, the implementation reads the flags and verifies that the encoded state is supported and permitted by the current runtime policy. The flags described in this section constitute the initially implemented DEK+ header flags set. As additional flags are deemed necessary, this section will be updated to reflect any revisions or additions. A `flags` value of zero represents the normal initial implementation password-protected form in which the DEK is wrapped and authenticated using KEK material derived from a non-empty password.

```

/**** Empty Password Support ****/
/*
 *   Indicates that the KEK is derived from the empty password ("").
 *   Runtime requires `allow_empty_password` to be true.
 */
#define LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD    (1u << 0)

```

This flag indicates the DEK is wrapped and authenticated using KEK material derived from a valid zero-length password string (""). The DEK+ object continues to contain a KDF salt, initialization vector, wrapped DEK, and authentication tag. This flag does not indicate that the password-based protection has been removed; it indicates that the password input used for KEK derivation is the empty string. This flag is set automatically when a DEK+ object is created or rekeyed using an empty-string password and the operation is authorized by the runtime policy. When opening the object, the implementation must reject empty-password recovery unless both this flag and the corresponding runtime authorization is enabled. When this flag is set, the KDF and MAC fields retain their expected non-zero values when

processing a wrapped DEK. This flag should never be set when `LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK` is set.

```

/**** Cleartext DEK Support ****/
/*
 * Indicates no KEK exists (i.e., the wrapped_dek field stores a cleartext
 * DEK). The salt, IV, and MAC length fields are expected to be zero.
 * Runtime will require `allow_cleartext_dek` to be true.
 */
#define LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK (1u << 1)

```

This flag indicates that password-based protection has been removed and the DEK is stored directly within the DEK+ object without deriving a KEK. For this representation, password-derived KEK material, a KDF salt, an initialization vector, and a wrapped DEK ciphertext are not used. This flag is set automatically by an authorized Remove Password operation. It must not be set merely because the user provided an empty-string password. When opening the object, the implementation must reject cleartext DEK+ recovery unless both this flag and the corresponding runtime authorization is enabled. This flag should never be set when `LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD` is set. When this flag is set, `kdf_id`, `kdf_n`, `kdf_r`, `kdf_p`, `kdf_out_len`, `cipher_id`, `cipher_mode`, `mac_id`, `salt_len`, `iv_len`, and `mac_len` are expected to be zero. Additionally, `dek_payload_len` is expected to be 32.

To validate the existing flags, the known bit mask should be defined and then validated. This may take the form of:

```

/* Define the known bit mask */
#define LFBT_DEKPLUS_FLAGS_ALL_KNOWN \
    (LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD | \
     LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK)

/* Validate the DEK+ Header Flags */
if (hdr->flags & ~LFBT_DEKPLUS_FLAGS_ALL_KNOWN)
    HGOTO_ERROR(..., "unsupported DEK+ header flag");

```

Unknown flags will cause rejection of the DEK+ object unless those bits are explicitly documented as something that may be ignored.

5.2.4 Serialization and Endianness

The initial version of the DEK+ format serializes all multi-byte integer fields into little-endian byte order. This applies to flags, algorithm identifiers, KDF parameters, and length fields. Byte-array fields such as magic, reserved, KDF salt, wrapped DEK, and authentication tag are written exactly as byte sequences and do not require endian conversion. Read/write helpers should explicitly encode and decode all fixed-width integer fields according to the DEK+ little-endian on-disk representation. Doing

so avoids relying on host-native structure layout and keeps the DEK+ format portable across supported platforms.

5.2.5 Cryptographic Construction

The initial implementation uses an Encrypt-then-MAC construction. An Encrypt-then-MAC construction is a cryptographic design pattern in which the plaintext is first encrypted to produce ciphertext, after which a separate authentication tag is computed over the resulting ciphertext and associated metadata. During recovery, the authentication tag is verified before any decryption operation is attempted. This ordering ensures that corrupted or maliciously modified ciphertext is detected prior to decryption and is widely regarded as the preferred composition method when confidentiality and integrity are provided by separate cryptographic primitives. The DEK is wrapped using AES-256-CBC. Both integrity and authenticity are provided through an HMAC-SHA256 authentication tag. The authentication tag is computed over the DEK+ header, KDF salt, IV, and wrapped DEK prior to being written to disk. During recovery, the authentication tag is verified before the wrapped DEK is decrypted.

The KDF salt should be generated with secure random number generation when the DEK+ object is created. Salt is not considered secret or privileged information and must be stored in the DEK+ object as it is a necessary component for reconstructing the KEK from the user's password. The length of the KDF salt, in bytes, depends on the underlying cryptographic algorithm (for our purposes, 16 bytes is appropriate). Regarding the actual KDF function selection, `libcrypt` supports both `PBKDF2` and `scrypt` as configurable-cost password-based KDFs. Between these two options, `scrypt` provides stronger resistance to offline password-guessing attacks because each derivation requires a configurable amount of memory⁹. This increases the memory cost of deriving the KEK material; however, the selected parameters are expected to require memory on the order of tens of megabytes per derivation¹⁰. The DEK is a 32-byte binary AES-256 key. The KDF produces 64 bytes of KEK material using `scrypt` with $N = 32768$, $r = 8$, and $p = 1$. The first 32 bytes (the wrapping key) are used as the AES-256-CBC wrapping key, and the second 32 bytes (the authentication key) are used as the HMAC-SHA256 key for encrypt-then-MAC authentication.

The Initialization Vector (IV) used to encrypt the DEK should also be stored in the DEK+ object. Like the salt, the IV is not a secret value, but it does need to be unique for the encryption operation under the derived KEK. A new IV should be used for each individual encryption operation. The size of the IV depends on the encryption mode selected. Since AES-256-CBC is already utilized in the Encryption VFD, it will be used for the initial implementation, requiring the IV to be 16 bytes.

⁹ If an adversary obtains a DEK+ object, offline password guessing becomes the primary method of attempting to recover the wrapped DEK. Consequently, both the strength of the user-selected password and the computational and memory cost imposed by the KDF directly affect the design's resistance to offline password-guessing attacks.

¹⁰ PBKDF2 is a well-established password-based KDF, but its cost is primarily computational and is commonly increased by raising the iteration count. `Scrypt` adds a configurable memory requirement to its computational cost, increasing the resource cost of large-scale password-guessing attacks implemented with GPUs, ASICs, or other specialized hardware. For this reason, `scrypt` was selected as the default KDF for the initial implementation.

Although many binary file formats employ checksums to detect accidental corruption, a separate checksum is intentionally omitted from the DEK+ format. The HMAC-SHA256 authentication tag already provides integrity verification for the authenticated contents of the file, detecting both accidental corruption and intentional modification. Since an attacker capable of modifying the DEK+ object can trivially recompute an unencrypted checksum but cannot recompute a valid HMAC without knowledge of the derived authentication key, the HMAC subsumes the role traditionally served by a checksum while providing substantially stronger security guarantees. In fact, the HMAC simultaneously provides integrity and authenticity.

5.2.6 Secret and Non-Secret Material

The DEK+ format intentionally distinguishes between information that must remain confidential and information that is required only to interpret or validate the DEK+ object. This distinction influences both the on-disk representation and the application's memory-management strategy. The following values are not considered secret and may be stored directly within the DEK+ object and processed using ordinary application memory:

- DEK+ header
- KDF parameters
- KDF salt
- IV
- wrapped DEK
- authentication tag

Ordinary serialized metadata may be staged in conventional application memory since the disclosure of these values does not compromise the confidentiality of the DEK. Although the wrapped DEK contains confidential information in the cryptographic sense, it is already encrypted and therefore does not require protection by the `libgcrypt` secure memory pool. In contrast, the following values are considered secret material and should be allocated from the `libgcrypt` secure memory pool:

- User password (while being processed)
- Derived KEK material
- Recovered plaintext DEK

The user-provided password and derived KEK only exist for the duration required to authenticate the DEK+ object and recover or rewrap the DEK and should be securely erased immediately after use. The recovered DEK remains resident in the secure memory pool for the lifetime of the associated encrypted HDF5 file (or Encryption VFD instance). Ownership of the recovered DEK is transferred to the component responsible for maintaining the active Encryption VFD instance. Applications interacting through the public API are not expected to manage the lifetime of this allocation directly unless explicitly outlined for a particular operation.

5.3 Code Components

The key management implementation will be provided by the standalone `LFBT_keymgmt` component. Application-facing operations will be exposed through the public API functions that will accept an HDF5 file identifier (`hid_t`) when the operation targets an open encrypted HDF5 file. The HDF5-facing wrappers will validate the file identifier and route the request to the appropriate internal Encryption VFD instance.

The general internal control interface mechanism is expected to follow the pattern:

```
herr_t H5FDctl(
    H5FD_t *file,
    uint64_t op_code,
    uint64_t flags,
    const void *input,
    void **output
);
```

The `op_code` value identifies the requested key management operation. The `flags` argument provides VFD-specific control semantics defined by the broader `H5FDctl()` interface. DEK+ specific flags are expected to be passed as part of the input structure (i.e., they are expected to be present within `LFBT_keymgmt_args_t::operation_flags`). The `input` pointer references an operation-specific input structure, and the `output` pointer may be used to return operation-specific output, such as a pointer to a recovered DEK in the `libgcrypt` secure memory pool.

5.3.1 Public LFBT_keymgmt Operations

The public `LFBT_keymgmt` API provides application-facing entry points for creating, recovering, validating, and modifying Encryption VFD key material. Operations that act on the DEK+ object embedded within an open encrypted HDF5 file accept an HDF5 file identifier (`hid_t`). The public API does not expose `H5FD_t`, `H5F_t`, `H5VL_object_t`, or any other internal HDF5 structure.

Public operations that act on an open encrypted HDF5 file will accept an HDF5 file identifier (`hid_t`) provided by the caller. The public wrapper validates the supplied identifier and routes the request through the applicable HDF5 file, VOL, and VFD layers. The internal adapter may ultimately invoke `H5FDctl()` using the active `H5FD_t *`, but the mechanism used to reach the VFD remains an implementation detail. The standalone `LFBT_keymgmt` component remains independent of the HDF5 internal data structures and operates only through the abstract interfaces supplied by the caller. The initial public operation set is expected to include the following functions:

```
herr_t LFBT_keymgmt_create( hid_t file_id, const LFBT_keymgmt_args_t *args,
    LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_unlock( hid_t file_id, const LFBT_keymgmt_args_t *args,
    LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_validate( hid_t file_id, const LFBT_keymgmt_args_t *args,
    LFBT_keymgmt_result_t *result );
```

```

herr_t LFBT_keymgmt_change_password( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_remove_password( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_add_password( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_update_kdf( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_migrate( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_import_dek( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_export_dek( hid_t file_id, const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

```

Function-specific argument requirements are described through the `LFBT_keymgmt_args_t` and `LFBT_keymgmt_result_t` field documentation and relevant internal function documentation below. Operations that do not require an open encrypted HDF5 file should use standalone entry points that omit the `file_id` field. Examples include generating an independent DEK and querying the capabilities compiled into the `LFBT_keymgmt` component:

```

herr_t LFBT_keymgmt_generate_dek( const LFBT_keymgmt_args_t *args,
LFBT_keymgmt_result_t *result );

herr_t LFBT_keymgmt_get_capabilities( LFBT_keymgmt_capabilities_t *capabilities );

```

A public function that accepts the `file_id` field should reject the request when:

- `file_id` is not a valid open HDF5 file identifier;
- the active VOL connector does not support the required optional operation;
- the active VFD is not the Encryption VFD or does not implement the requested control operation;
- the file was opened without the access required by a modifying operation;
- the provided argument or result structure version is unsupported; or
- the requested capability is absent from the current build.

The public wrapper is responsible for application-facing argument validation, VOL dispatch, and translation of errors into the public HDF5 error stack. Cryptographic processing, DEK+ serialization, password policy enforcement, and storage access remain the responsibility of the Encryption VFD and standalone `LFBT_keymgmt` implementation. `LFBT_keymgmt_capabilities_t` reports the security-sensitive functionality present in the current build. The capability flags are derived exclusively

from the component's compiled configuration. They do not describe the state of a particular DEK+ object, do not represent operation flags, and do not authorize a caller to use the reported functionality. `LFBT_keymgmt_get_capabilities()` does not require an open HDF5 file because the reported values describe the `LFBT_keymgmt` implementation rather than an individual Encryption VFD instance. A future implementation may separately provide per-file or per-VFD capability discovery when support depends on the active driver or storage backend.

```
#define LFBT_KEYMGMT_CAP_EMPTY_PASSWORD      (1ULL << 0)
#define LFBT_KEYMGMT_CAP_CLEARTEXT_DEK      (1ULL << 1)
#define LFBT_KEYMGMT_CAP_CLEARTEXT_EXPORT   (1ULL << 2)
#define LFBT_KEYMGMT_CAP_EXTERNAL_DEKPLUS   (1ULL << 3)

#define LFBT_KEYMGMT_CAP_ALL_KNOWN           \
    (LFBT_KEYMGMT_CAP_EMPTY_PASSWORD |      \
    LFBT_KEYMGMT_CAP_CLEARTEXT_DEK |        \
    LFBT_KEYMGMT_CAP_CLEARTEXT_EXPORT |     \
    LFBT_KEYMGMT_CAP_EXTERNAL_DEKPLUS)

typedef struct LFBT_keymgmt_capabilities_t {
    size_t struct_size;
    uint32_t struct_version;
    uint64_t capability_flags;
} LFBT_keymgmt_capabilities_t;
```

5.3.2 Runtime Policy and Operation Flags

Runtime policy and operation flags control the behavior of an individual key management request. Unlike the flags stored in `LFBT_dekplus_hdr_t`, runtime flags are not serialized into the DEK+ object and do not describe its persistent cryptographic representation. Instead, they indicate which actions the current caller requests and which security-sensitive behaviors the current execution environment permits. The runtime controls are provided through the `LFBT_keymgmt_args_t` structure. Operation flags describe procedural behavior, such as whether terminal prompting is permitted, whether an existing DEK+ object should be overwritten, or whether a non-interactive caller has explicitly acknowledged a security-reducing operation. Runtime policy fields, such as `allow_empty_password` and `allow_cleartext_dek`, authorize specific cryptographic representations or transitions. A runtime authorization does not by itself modify the DEK+ header; the implementation derives the resulting serialized flags only after validating the requested operation and after successfully rewriting the DEK+ object. Runtime controls must be validated independently from the serialized header flags. A serialized flag identifies the cryptographic representation used by an existing DEK+ object, while the corresponding runtime authorization determines whether the current process is permitted to use or produce that representation. When both are required, the operation must fail unless the serialized state, runtime authorization, and build-time capabilities are mutually consistent.

Currently, several operation flags will be utilized. These flags are defined and discussed below.

```
/** Bypass Terminal-Based Password Prompt Support */
```

```

/*
 * Indicates that terminal-based password collection should be bypassed. Any
 * password required by the selected operation must be supplied
 * programmatically through LFBT_keymgmt_args_t.
 */

#define LFBT_KEYMGMT_OP_BYPASS_PASSWORD_PROMPT (1ULL << 0)

```

This runtime operation flag requires the key management workflow to proceed without displaying an interactive terminal-based password prompt. It is used for programmatic workflows in which all required password input has already been provided through `LFBT_keymgmt_args_t`, or when the DEK+ object does not require a non-empty password. This flag does not authorize empty-string password usage or cleartext DEK representation. Empty-string password processing requires the applicable build-time capability, the `allow_empty_password` runtime authorization, and when reading an existing DEK+ object, the `LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD` header flag. Cleartext DEK processing similarly requires its applicable build-time capability, `allow_cleartext_dek`, and, when reading an existing DEK+ object, `LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK`. The implementation must reject the operation if bypassing the prompt would leave the required authentication input unavailable. For example, a password-protected DEK+ object cannot be unlocked with this flag set unless a valid password is provided programmatically. This flag only affects terminal-based password collection. It does not suppress mandatory security warnings or act as a substitute for the explicit acknowledgment of a security-reducing operation.

```

#define LFBT_KEYMGMT_OP_CONFIRM_SECURITY_REDUCTION (1ULL << 1)

```

This runtime operation flag records the caller's explicit acknowledgement that the requested operation reduces the protection applied to the DEK. It is required for non-interactive operations that remove password-based protection, create or retain a cleartext DEK representation, export a cleartext DEK, or otherwise transition key material from a more protected representation into a less protected representation. For interactive workflows, the implementation should obtain equivalent confirmation directly from the user after displaying a clear description of the security consequences. The implementation may then set this flag before invoking the underlying shared implementation. For non-interactive workflows, this flag must be provided explicitly by the calling application or derived from an equally explicit API argument.

This flag records acknowledgment. It does not enable the underlying capabilities. The requested functionality must also be present in the build, permitted by the runtime policy fields, and be valid for the selected operation code. Setting this flag alone must never permit password removal, cleartext storage, or DEK export. This flag is not required for operations that preserve or strengthen the existing security model, such as changing from one non-empty password to another, adding password protection to a cleartext DEK+ object, or increasing the KDF work factor.

```

/**** External DEK+ File Support (For Future Development) ****/
/*
 *   Intended for future use. Will indicate that a complete DEK+ object should be
 *   written to an external destination.
 */
#define LFBT_KEYMGMT_OP_EXTERNAL_COPY          (1ULL << 4)

```

This runtime operation flag requests that the operation create a complete external DEK+ object in addition to, or instead of, updating the embedded DEK+ object associated with the encrypted HDF5 file. The external object may be used as a backup, recovery copy, or independently password-protected duplicate of the same underlying DEK. The external DEK+ object is a complete, serialized representation rather than a reference to the embedded DEK+ object. It therefore contains its own header, KDF parameters, KDF salt, initialization vector, DEK payload, and authentication tag. When the external copy is protected with a different password, the implementation must recover the existing DEK into the `libgcrypt` secure memory pool and create new KDF salt, IV, KEK material, wrapped DEK, and authentication tag for the external destination. The encrypted HDF5 data and underlying DEK remain unchanged. The caller must provide a valid `destination_path`. If the destination already exists, replacement is permitted only when `allow_overwrite` is true. The `LFBT_KEYMGMT_OP_ATOMIC_REPLACE` operation flag may additionally be provided to require atomic replacement of an existing external object. This operation flag does not alter `LFBT_dekplus_hdr_t::flags`; the DEK+ object storage location does not alter the cryptographic interpretation of the underlying DEK+ object. It also does not authorize password removal, cleartext DEK storage, or plaintext DEK export. Any external copy must independently satisfy the build-time and runtime policies applicable to its selected representation.

To validate the existing operational flags, the known bit mask should be defined and then validated. This may take the form of:

```

#define LFBT_KEYMGMT_OP_ALL_KNOWN \
    (LFBT_KEYMGMT_OP_BYPASS_PASSWORD_PROMPT | \
     LFBT_KEYMGMT_OP_CONFIRM_SECURITY_REDUCTION | \
     LFBT_KEYMGMT_OP_ATOMIC_REPLACE | \
     LFBT_KEYMGMT_OP_RETURN_DEK | \
     LFBT_KEYMGMT_OP_EXTERNAL_COPY)

if (args->operation_flags & ~LFBT_KEYMGMT_OP_ALL_KNOWN)
    HGOTO_ERROR(..., "unsupported key-management operation flag");

```

5.3.3 Internal HDF5/VFD Control Dispatch

The control codes defined in this subsection are internal dispatch identifiers used by the HDF5/VFD adapter. They are not public `LFBT_keymgmt` API identifiers and are not constructed directly by

ordinary applications. Public wrapper functions validate the application-facing arguments and translate the requested operation into the corresponding internal VFD control request.

The public DEK+ operations should be represented as `H5FDctl` operation codes. These operation codes should mirror the supported operations described in Section 2.1 so that the Encryption VFD and `h5keytools` use the same conceptual operation model. The initial operation set should include:

```
#define H5FD_CTL_KEYMGMT_DEKPLUS_CREATE /* Create DEK+ object */
#define H5FD_CTL_KEYMGMT_DEKPLUS_UNLOCK /* Recover DEK from DEK+ object */
#define H5FD_CTL_KEYMGMT_DEKPLUS_VALIDATE /* Validate DEK+ object */
#define H5FD_CTL_KEYMGMT_DEKPLUS_CHANGE_PASSWORD /* Change password / rotate KEK */
#define H5FD_CTL_KEYMGMT_DEKPLUS_REMOVE_PASSWORD /* Convert to cleartext DEK representation */
#define H5FD_CTL_KEYMGMT_DEKPLUS_ADD_PASSWORD /* Convert from cleartext DEK representation */
#define H5FD_CTL_KEYMGMT_DEKPLUS_UPDATE_KDF /* Rewrap using updated KDF parameters */
#define H5FD_CTL_KEYMGMT_DEKPLUS_MIGRATE /* Migrate to future DEK+ format */
#define H5FD_CTL_KEYMGMT_DEKPLUS_EXPORT_DEK /* Export DEK to cleartext mechanism */
#define H5FD_CTL_KEYMGMT_DEKPLUS_IMPORT_DEK /* Import cleartext DEK into DEK+ */
```

For normal Encryption VFD operation, the two primary control operations are:

```
H5FDctl(file, H5FD_CTL_KEYMGMT_DEKPLUS_CREATE, flags, input, output);
H5FDctl(file, H5FD_CTL_KEYMGMT_DEKPLUS_UNLOCK, flags, input, output);
```

`H5FD_CTL_KEYMGMT_DEKPLUS_CREATE` is used when a new encrypted HDF5 file requires the creation of an embedded DEK+ object. `H5FD_CTL_KEYMGMT_DEKPLUS_UNLOCK` is used when opening an existing encrypted HDF5 file that is already associated with a DEK+ object. Other operations, such as password changes, password removal, KDF updates, format migration, import, and export, are expected to be invoked primarily through the `h5keytools` utility. These operations should still use the same underlying control operation model and internal helper functions, but they should not be part of the normal Encryption VFD open path.

5.3.4 Internal High-Level DEK+ Operations

The internal DEK+ operations provide the primary implementation entry points used by both the Encryption VFD and the `h5keytools` companion utility. Unlike the lower-level helper routines described in later sections, these functions orchestrate complete key management workflows by coordinating header initialization, password validation, cryptographic processing, secure-memory allocation, and DEK+ serialization. Each operation is responsible for invoking the appropriate internal helper functions while maintaining consistent error handling and cleanup semantics.

```
herr_t LFBT__keymgmt_dekplus_create(LFBT_keymgmt_io_t *io, const
LFBT_keymgmt_args_t *args, uint8_t **out_secure_dek);
```

Creates a new DEK+ object. When `allow_existing_dek` is false, a new 32-byte DEK is generated using the `libgcrypt` random-number generator and is allocated from the `libgcrypt` secure memory pool. Otherwise, the user-provided DEK is copied into the secure memory pool and used directly. The provided password is validated against the current password policy, 64 bytes of KEK material are derived using the configured Key Derivation Function, the DEK is wrapped using AES-256-CBC, an HMAC-SHA256 authentication tag is computed, and the resulting DEK+ object is written to persistent storage. Upon successful completion, the function returns a pointer to the recovered plaintext DEK residing within the `libgcrypt` secure memory pool. Operation with an empty-string password should be permitted only when explicitly requested through the `allow_empty_password` flag, and only when working on a trusted, isolated system or in a non-interactive execution environment where a password prompt is impractical. Because an empty password-derived KEK provides no password entropy, the DEK+ object created under these circumstances should not be considered secure. It may subsequently be rekeyed using `h5keytools` to restore password-based protection. This routine is primarily used when creating a new encrypted HDF5 file.

```
herr_t LFBT__keymgmt_dekplus_unlock(LFBT_keymgmt_io_t *io, const
LFBT_keymgmt_args_t *args, uint8_t **out_secure_dek);
```

Reads the fixed header and payload from the configured DEK+ storage location, validates the header, reconstructs the KEK material using the configured Key Derivation Function, verifies the HMAC-SHA256 authentication tag, decrypts the wrapped DEK, allocates secure memory for the recovered plaintext DEK, and returns a pointer to the recovered 32-byte binary DEK stored in the `libgcrypt` secure memory pool. The user-provided password and KEK material are securely erased before returning to the caller. Primarily used when opening an existing encrypted HDF5 file.

5.3.5 Omnibus Input and Output Structures

Each `H5FDctl` operation should define a single omnibus input structure rather than passing function parameters independently. This keeps the public interface stable while allowing future fields to be added without changing the control function signature. Each field in this structure will explicitly state which functions require a non-zero value.

```
/******
 *
 * Structure: LFBT_keymgmt_args_t
 *
 * Purpose:
 *
 * Provides the omnibus input arguments used by public LFBT_keymgmt
 * operations. The same structure is accepted by DEK+ creation, recovery,
 * validation, password-management, migration, import, and export
 * operations so that the public interface can be extended without
```

```

*   changing each function signature.
*
*   Fields that are not applicable to the selected operation must be set to
*   zero, false, or NULL, as appropriate. Before performing storage access
*   or cryptographic processing, the implementation must validate all
*   required fields and reject contradictory, unsupported, or unexpectedly
*   populated fields when their presence could make the requested behavior
*   ambiguous.
*
*   Password fields point to null-terminated strings. A valid pointer to a
*   zero-length string represents an empty-string password. A NULL password
*   pointer means that no password was supplied programmatically and may
*   cause an interactive operation to prompt the user. NULL must not be
*   interpreted as an empty-string password.
*
*   Secret input buffers, including password, new_password, and existing_dek,
*   remain owned by the caller. The implementation must not modify or free
*   these buffers. When secret material must be retained or processed
*   internally, it must be copied into an appropriately managed libgcrypt
*   secure-memory allocation.
*
* Fields:
*
*   size_t struct_size:
*       Size, in bytes, of the caller-provided structure. The caller must
*       initialize this field to sizeof(LFBT_keymgmt_args_t). The
*       implementation uses this value to validate structure compatibility
*       and to permit future versions to append fields without changing the
*       meaning of the existing layout.
*
*       Required for:
*           All operations accepting LFBT_keymgmt_args_t.
*
*   uint32_t struct_version:
*       Version of the public argument structure. The initial implementation
*       uses a value of 1. The implementation must reject unsupported
*       structure versions before examining version-dependent fields.
*
*       Required for:
*           All operations accepting LFBT_keymgmt_args_t.
*
*   const char *source_path:
*       Path to an external source representation. This field identifies a
*       source that is not already represented by the supplied file_id or
*       by the caller-provided existing_dek buffer. It may refer to an
*       external DEK+ object, a cleartext DEK file, or another supported
*       source representation.
*
*       Non-NULL for:
*           Operations that read an external source, including standalone
*           UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*           ADD_PASSWORD, UPDATE_KDF, MIGRATE, and EXPORT_DEK operations.
*
*       Optional for:
*           IMPORT_DEK when the imported DEK is instead supplied through
*           existing_dek.

```

```

*      NULL for:
*          Operations whose source is the embedded DEK+ object associated
*          with file_id, and GENERATE_DEK unless the operation explicitly
*          consumes an existing external source.
*
*      const char *destination_path:
*          Path to an external destination representation. This field
*          identifies the location to which a new DEK, DEK+ object, exported
*          DEK, backup copy, or migrated representation will be written.
*
*      Non-NULL for:
*          Operations that create an external destination, including
*          standalone CREATE, GENERATE_DEK, MIGRATE, EXPORT_DEK, and
*          IMPORT_DEK operations.
*
*      Optional for:
*          CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD, and UPDATE_KDF
*          when the result is written to a separate destination rather
*          than replacing the source object in place.
*
*      NULL for:
*          Operations that rewrite the embedded DEK+ object associated
*          with file_id, UNLOCK, and validation-only operations.
*
*      uint64_t operation_flags:
*          Bitmask of per-invocation procedural controls. These flags may
*          control terminal prompting, security-reduction acknowledgement,
*          atomic replacement, DEK return behavior, creation of an external
*          copy, and other operation-specific behavior.
*
*          Operation flags do not describe compiled capabilities and are not
*          copied directly into LFBT_dekplus_hdr_t::flags. Any serialized DEK+
*          header flags are derived separately after the requested operation,
*          runtime policy, compiled capabilities, and resulting representation
*          have been validated.
*
*      May be non-zero for:
*          Any operation for which one or more defined procedural controls
*          are applicable.
*
*      const uint8_t *existing_dek:
*          Pointer to a caller-provided plaintext DEK. This field allows an
*          existing DEK to be imported or used instead of generating a new
*          random DEK. The pointed-to buffer remains owned by the caller and
*          must not be modified or freed by the implementation.
*
*      Non-NULL for:
*          CREATE when allow_existing_dek is true;
*          IMPORT_DEK when the DEK is supplied directly in memory.
*
*      NULL for:
*          UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*          ADD_PASSWORD, UPDATE_KDF, and ordinary MIGRATE operations.
*
*      size_t existing_dek_len:
*          Length, in bytes, of the buffer referenced by existing_dek. Version 1
*          requires a value of 32 whenever existing_dek is non-NULL.

```

```

*
*      Non-zero for:
*          CREATE or IMPORT_DEK when existing_dek is non-NULL.
*
*      Zero for:
*          Operations that do not use existing_dek.
*
*      const char *password:
*          Current or initial password used to create, authenticate, or recover
*          a password-protected DEK+ object. A valid zero-length string
*          represents an empty-string password and is distinct from NULL.
*
*      Non-NULL for non-interactive:
*          CREATE, UNLOCK, CHANGE_PASSWORD, REMOVE_PASSWORD, UPDATE_KDF,
*          MIGRATE, EXPORT_DEK, and authenticated VALIDATE operations.
*
*      Optional for:
*          Interactive forms of those operations, where NULL permits the
*          front-end to obtain the password through an approved terminal
*          input mechanism.
*
*      Normally NULL for:
*          GENERATE_DEK;
*          ADD_PASSWORD when the source object contains a cleartext DEK and
*          no current password is required.
*
*      const char *new_password:
*          Replacement password used to establish new password-derived KEK
*          protection for an existing DEK. A valid zero-length string requests
*          an empty-string password and therefore also requires the applicable
*          compiled capability and allow_empty_password authorization.
*
*      Non-NULL for non-interactive:
*          CHANGE_PASSWORD and ADD_PASSWORD.
*
*      Optional for:
*          Interactive CHANGE_PASSWORD and ADD_PASSWORD operations, where
*          NULL permits the front-end to prompt for the replacement
*          password and any required confirmation.
*
*      NULL for:
*          CREATE, UNLOCK, VALIDATE, REMOVE_PASSWORD, GENERATE_DEK,
*          EXPORT_DEK, and operations that do not establish a replacement
*          password.
*
*      bool allow_empty_password:
*          Runtime authorization permitting use or creation of a wrapped DEK
*          representation whose KEK is derived from a zero-length password.
*          This field does not request password removal and does not authorize
*          cleartext DEK storage.
*
*      True only for:
*          CREATE, CHANGE_PASSWORD, or ADD_PASSWORD when an empty-string
*          password is intentionally requested;
*          UNLOCK or authenticated VALIDATE when processing a DEK+ object
*          carrying LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD.
*

```

```

*      False for:
*          Ordinary non-empty-password operations and REMOVE_PASSWORD.
*
*  bool allow_cleartext_dek:
*      Runtime authorization permitting the implementation to read, create,
*      or preserve a DEK+ object whose DEK payload is stored in cleartext
*      and whose header carries LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK.
*
*      This field authorizes the representation only. It does not by itself
*      request password removal, DEK export, or another security-reducing
*      operation. Any required operation flag and explicit acknowledgement
*      must also be present.
*
*      True only for:
*          REMOVE_PASSWORD;
*          ADD_PASSWORD when the source object stores a cleartext DEK;
*          UNLOCK or VALIDATE when opening a cleartext-DEK object;
*          MIGRATE, IMPORT_DEK, or EXPORT_DEK when the source or
*          destination representation requires cleartext-DEK support.
*
*      False for:
*          Normal password-protected and empty-string-password operations
*          that do not read or produce a cleartext DEK representation.
*
*  bool allow_existing_dek:
*      Indicates that CREATE may use the caller-provided existing_dek
*      instead of generating a new random DEK. When true, existing_dek must
*      be non-NULL and existing_dek_len must contain the required DEK
*      length.
*
*      True only for:
*          CREATE when a valid caller-provided DEK is intentionally used.
*
*      False for:
*          CREATE when a new random DEK should be generated, and all other
*          operations.
*
*  bool allow_overwrite:
*      Explicitly authorizes replacement of an existing destination object
*      or file. When false, an operation that would replace an existing
*      destination must fail before modifying that destination.
*
*      This field authorizes overwrite behavior but does not guarantee
*      atomic replacement. Atomic replacement requires the corresponding
*      operation flag and support from the selected I/O backend.
*
*      May be true for:
*          CREATE, GENERATE_DEK, CHANGE_PASSWORD, REMOVE_PASSWORD,
*          ADD_PASSWORD, UPDATE_KDF, MIGRATE, EXPORT_DEK, and IMPORT_DEK
*          when an existing destination may be replaced.
*
*      False for:
*          UNLOCK, VALIDATE, and operations that do not write to an
*          existing destination.
*
*  uint32_t source_format:
*      Identifier for the DEK handling mechanism used by the source input.

```

```

*      This field allows the implementation to distinguish among a DEK+
*      object, a cleartext binary DEK, or another supported representation
*      when the source format cannot be inferred safely from the selected
*      operation or storage context.
*
*      Non-zero for:
*          MIGRATE, IMPORT_DEK, and EXPORT_DEK when the source mechanism
*          cannot be inferred unambiguously.
*
*      Zero for:
*          Operations whose source is known to be a DEK+ object or whose
*          source representation is otherwise unambiguous.
*
uint32_t target_format:
*      Identifier for the requested destination DEK handling mechanism.
*      This field determines the representation to be produced by an
*      operation that supports more than one output mechanism.
*
*      Non-zero for:
*          MIGRATE, IMPORT_DEK, and EXPORT_DEK.
*
*      Zero for:
*          CREATE, UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*          ADD_PASSWORD, and UPDATE_KDF when the resulting representation
*          is unambiguously DEK+.
*
uint32_t kdf_id:
*      Optional identifier for the password-based KDF to use when writing a
*      new wrapped-DEK representation. Version 1 supports scrypt.
*
*      May be non-zero for:
*          CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*          and IMPORT_DEK.
*
*      Zero means:
*          Use the Version 1 default for a newly created representation or,
*          when permitted by the operation, retain the validated existing
*          value.
*
uint32_t kdf_n:
*      Optional scrypt CPU and memory cost parameter, N. A non-zero value
*      requests an explicit override and must satisfy the implementation's
*      supported range and power-of-two requirements.
*
*      May be non-zero for:
*          CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*          and IMPORT_DEK.
*
*      Zero means:
*          Use the Version 1 default for a new representation or retain the
*          validated existing value when permitted.
*
uint32_t kdf_r:
*      Optional scrypt block-size parameter, r. A non-zero value requests
*      an explicit override and must satisfy the implementation's supported
*      parameter limits.
*

```

```

*         May be non-zero for:
*             CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*             and IMPORT_DEK.
*
*         Zero means:
*             Use the Version 1 default for a new representation or retain the
*             validated existing value when permitted.
*
* uint32_t kdf_p:
*     Optional scrypt parallelization parameter, p. A non-zero value
*     requests an explicit override and must satisfy the implementation's
*     supported parameter limits.
*
*         May be non-zero for:
*             CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*             and IMPORT_DEK.
*
*         Zero means:
*             Use the Version 1 default for a new representation or retain the
*             validated existing value when permitted.
*
* uint32_t kdf_out_len:
*     Optional length, in bytes, of the material produced by the KDF.
*     Version 1 normally requires 64 bytes: 32 bytes for the AES-256-CBC
*     wrapping key and 32 bytes for the HMAC-SHA256 authentication key.
*     Unsupported non-zero values must be rejected.
*
*         May be non-zero for:
*             CREATE, CHANGE_PASSWORD, ADD_PASSWORD, UPDATE_KDF, MIGRATE,
*             and IMPORT_DEK.
*
*         Zero means:
*             Use the Version 1 default for a new representation or retain the
*             validated existing value when permitted.
*
*****/
typedef struct LFBT_keymgmt_args_t {
    size_t    struct_size;
    uint32_t  struct_version;

    const char *source_path;
    const char *destination_path;

    uint64_t  operation_flags;

    const uint8_t *existing_dek;
    size_t      existing_dek_len;

    const char *password;
    const char *new_password;

    bool allow_empty_password;
    bool allow_cleartext_dek;
    bool allow_existing_dek;
    bool allow_overwrite;

    uint32_t  source_format;

```

```

uint32_t target_format;

uint32_t kdf_id;
uint32_t kdf_n;
uint32_t kdf_r;
uint32_t kdf_p;
uint32_t kdf_out_len;
} LFBT_keymgmt_args_t;

```

Similarly, DEK+ operations should return a single omnibus structure rather than independent output values.

```

/*****
 *
 * Structure: LFBT_keymgmt_result_t
 *
 * Purpose:
 *
 * Provides the omnibus result returned by LFBT_keymgmt operations. The
 * same result structure is used for creation, recovery, validation,
 * password-management, migration, import, and export operations.
 *
 * Fields that are not applicable to the completed operation are returned
 * as zero, false, or NULL. A successful H5FDctl() return value indicates
 * that the requested operation completed successfully; the validation
 * fields provide additional detail for operations that distinguish
 * structural validation from authentication validation.
 *
 * When secure_dek is non-NULL, it points to secret material allocated from
 * the libgcrypt secure memory pool. Ownership is transferred to the component
 * that invokes the underlying LFBT_keymgmt operation. For operations routed
 * through an active Encryption VFD instance, the Encryption VFD assumes
 * ownership and retains the DEK for the lifetime of the open encrypted HDF5
 * file. An application using the public API does not directly own or release
 * this allocation unless a particular operation explicitly documents
 * application ownership.
 *
 * Fields:
 *
 * size_t struct_size:
 *     Size, in bytes, of the result structure populated by the
 *     implementation. Set to sizeof(LFBT_keymgmt_result_t) for every
 *     successfully initialized result.
 *
 *     Non-zero for:
 *         All operations returning this structure.
 *
 * uint32_t struct_version:
 *     Version of the public result structure. The initial value is 1.
 *
 *     Non-zero for:
 *         All operations returning this structure.
 *
 * uint8_t *secure_dek:

```

```

*      Pointer to the recovered, generated, or imported plaintext DEK stored
*      in the libgcrypt secure memory pool.
*
*      Non-NULL for:
*          CREATE, UNLOCK, GENERATE_DEK, IMPORT_DEK, and operations
*          explicitly requested to return the recovered DEK.
*
*      Optional for:
*          CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD, UPDATE_KDF,
*          MIGRATE, and EXPORT_DEK.
*
*      NULL for:
*          Structural validation that does not recover the DEK.
*
*      size_t secure_dek_len:
*          Length, in bytes, of secure_dek. Version 1 returns 32 whenever
*          secure_dek is non-NULL.
*
*      Non-zero when:
*          secure_dek is non-NULL.
*
*      Zero when:
*          secure_dek is NULL.
*
*      uint32_t detected_flags:
*          DEK+ header flags read from the source object or written to the
*          resulting destination object.
*
*      Meaningful for:
*          CREATE, UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
*          ADD_PASSWORD, UPDATE_KDF, MIGRATE, IMPORT_DEK, and EXPORT_DEK
*          when a DEK+ object is inspected or produced.
*
*      Zero for:
*          GENERATE_DEK and formats without DEK+ header flags.
*
*      uint32_t detected_version:
*          DEK+ format version read from or written to the relevant object.
*
*      Non-zero for:
*          Operations that inspect or produce a DEK+ object.
*
*      Zero for:
*          GENERATE_DEK and non-DEK+ representations.
*
*      uint32_t resulting_format:
*          Identifier for the DEK handling mechanism produced or detected by
*          the completed operation.
*
*      Non-zero for:
*          MIGRATE, IMPORT_DEK, EXPORT_DEK, and operations where the format
*          cannot otherwise be inferred.
*
*      May be zero for:
*          Operations that are unambiguously DEK+-specific.
*
*      bool structural_validation_performed:

```

```

*      Indicates whether the implementation examined the DEK+ header,
*      version, flags, algorithm identifiers, lengths, and total serialized
*      size.
*
*      True for:
*          VALIDATE, UNLOCK, password-management operations, UPDATE_KDF,
*          MIGRATE, and other operations that read an existing DEK+ object.
*
*      bool structural_validation_passed:
*          Indicates that structural validation completed successfully.
*          Meaningful only when structural_validation_performed is true.
*
*      bool authentication_performed:
*          Indicates whether the operation derived or otherwise obtained the
*          authentication key and verified the DEK+ authentication tag.
*
*          True for:
*              UNLOCK, authenticated VALIDATE, CHANGE_PASSWORD,
*              REMOVE_PASSWORD, UPDATE_KDF, MIGRATE, EXPORT_DEK, and other
*              operations that recover a wrapped DEK.
*
*          False for:
*              Structural-only validation and plaintext-DEK representations
*              that do not use password-derived authentication.
*
*      bool authentication_passed:
*          Indicates that authentication completed successfully. Meaningful
*          only when authentication_performed is true.
*
*      bool migration_required:
*          Indicates that the source object is structurally valid but requires
*          migration before the current implementation can use or rewrite it
*          in the requested form.
*
*          May be true for:
*              VALIDATE, UNLOCK, UPDATE_KDF, and MIGRATE.
*
*          False for:
*              Newly created Version 1 objects and operations unrelated to
*              format compatibility.
*
*****/
typedef struct LFBT_keymgmt_result_t {
    size_t    struct_size;
    uint32_t  struct_version;

    uint8_t   *secure_dek;
    size_t    secure_dek_len;

    uint32_t  detected_flags;
    uint32_t  detected_version;
    uint32_t  resulting_format;

    bool structural_validation_performed;
    bool structural_validation_passed;
    bool authentication_performed;
    bool authentication_passed;

```

```

    bool migration_required;
} LFBT_keymgmt_result_t;

```

5.3.6 LFBT_keymgmt I/O Abstraction

The standalone `LFBT_keymgmt` component must not depend on HDF5 file identifiers, `H5FD_t`, native file descriptors, `FILE *`, or a particular DEK+ storage location. Instead, the calling component supplies an abstract I/O context that provides the operations necessary to read and write a serialized DEK+ object. For embedded DEK+ operations, the Encryption VFD initializes the I/O context with callbacks that access the reserved DEK+ region of the encrypted HDF5 file. For standalone or future external-file operations, `h5keytools` initializes the same context with callbacks that access an external file or other supported storage representation. The callbacks must interpret offsets relative to the beginning of the DEK+ object rather than relative to the beginning of the containing HDF5 file or external storage object. The I/O context is an internal implementation structure and should be declared in the private `LFBT_keymgmt` header. It is not part of the ordinary application-facing API.

```

typedef herr_t (*LFBT_keymgmt_read_cb_t) (
    void      *context,
    uint64_t  offset,
    void      *buffer,
    size_t    size
);

typedef herr_t (*LFBT_keymgmt_write_cb_t) (
    void      *context,
    uint64_t   offset,
    const void *buffer,
    size_t     size
);

typedef herr_t (*LFBT_keymgmt_get_size_cb_t) (
    void      *context,
    uint64_t *size_out
);

typedef herr_t (*LFBT_keymgmt_set_size_cb_t) (
    void      *context,
    uint64_t size
);

typedef herr_t (*LFBT_keymgmt_flush_cb_t) (
    void *context
);

typedef herr_t (*LFBT_keymgmt_begin_update_cb_t) (
    void *context
);

typedef herr_t (*LFBT_keymgmt_commit_update_cb_t) (
    void *context
);

```

```

typedef herr_t (*LFBT_keymgmt_abort_update_cb_t) (
    void *context
);

/*****
 *
 * Structure: LFBT_keymgmt_io_t
 *
 * Purpose:
 *
 *   Describes the abstract storage interface through which the standalone
 *   LFBT_keymgmt component reads, writes, validates, and replaces serialized
 *   DEK+ objects.
 *
 *   The LFBT_keymgmt implementation must remain independent of HDF5 file
 *   identifiers, HDF5 internal structures, native file descriptors, FILE
 *   objects, and any particular DEK+ storage location. Instead, the calling
 *   component initializes this structure with an opaque context pointer and
 *   the callbacks required to access the selected storage backend.
 *
 *   For embedded DEK+ operations, the Encryption VFD may configure this
 *   structure with callbacks that access the reserved DEK+ region within an
 *   encrypted HDF5 file. For standalone or external-file operations,
 *   h5keytools may configure the same interface with callbacks that access a
 *   complete external DEK+ object. Future storage backends may provide other
 *   implementations without requiring changes to the core LFBT_keymgmt
 *   cryptographic or serialization logic.
 *
 *   Unless otherwise documented for a particular backend, all offsets
 *   supplied to the read and write callbacks are relative to the beginning
 *   of the serialized DEK+ object. They are not relative to the beginning of
 *   the containing HDF5 file, external file, device, or other storage
 *   container.
 *
 *   The callbacks and capability flags describe trusted implementation
 *   properties of the selected storage backend. They are distinct from the
 *   application-controlled operation arguments in LFBT_keymgmt_args_t.
 *   Runtime operation arguments describe what the caller requests, while
 *   this structure describes which storage operations the backend can
 *   perform.
 *
 *   The caller owns the LFBT_keymgmt_io_t structure, the opaque context
 *   referenced by context, and any resources used by the callbacks. The
 *   LFBT_keymgmt implementation must not free or replace those resources.
 *   The caller must ensure that the structure and its context remain valid
 *   for the duration of the key-management operation.
 *
 * Callback Semantics:
 *
 *   A callback that returns success must complete the entire requested
 *   operation. In particular, read and write callbacks must not report
 *   success after transferring fewer than the requested number of bytes.
 *
 *   Callback implementations must validate offsets, lengths, arithmetic,
 *   and backend boundaries before accessing storage. The LFBT_keymgmt

```

```

* implementation must also validate serialized DEK+ lengths before
* issuing payload reads or writes.
*
* Optional callbacks may be NULL only when their corresponding capability
* is not advertised and the selected operation does not require them.
* Required callbacks must be non-NULL and consistent with the advertised
* capability flags.
*
* Fields:
*
* size_t struct_size:
*     Size, in bytes, of the caller-provided structure. The caller must
*     initialize this field to sizeof(LFBT_keymgmt_io_t). The
*     implementation uses the value to validate structure compatibility
*     and to permit future versions to append fields without changing the
*     meaning of the existing layout.
*
*     Required for:
*         Every initialized LFBT_keymgmt_io_t instance.
*
* uint32_t struct_version:
*     Version of the abstract I/O interface. The initial implementation
*     uses a value of 1. The LFBT_keymgmt implementation must reject
*     unsupported versions before invoking any callback.
*
*     Required for:
*         Every initialized LFBT_keymgmt_io_t instance.
*
* void *context:
*     Opaque caller-owned pointer passed unchanged as the first argument
*     to every configured callback.
*
*     The interpretation of this pointer is defined entirely by the
*     storage backend. For the Encryption VFD, it may identify the active
*     VFD instance together with the location and size of the reserved
*     DEK+ region. For h5keytools, it may identify an external file handle,
*     path-specific state, temporary replacement file, or another
*     standalone storage object.
*
*     The LFBT_keymgmt implementation must not inspect, modify, or free
*     the object referenced by context.
*
*     Required for:
*         Backends whose callbacks require caller-specific state.
*
*     May be NULL for:
*         Stateless callback implementations that explicitly support a
*         NULL context.
*
* LFBT_keymgmt_read_cb_t read:
*     Callback that copies exactly size bytes from the serialized DEK+
*     object, beginning at the DEK+-relative offset, into buffer.
*
*     The callback must fail if the requested range lies outside the
*     readable DEK+ object extent, if the backend cannot complete the full
*     transfer, or if any supplied argument is invalid. A successful
*     return indicates that all requested bytes were copied.

```

*
 * Required for:
 * UNLOCK, VALIDATE, CHANGE_PASSWORD, REMOVE_PASSWORD,
 * ADD_PASSWORD, UPDATE_KDF, MIGRATE, EXPORT_DEK, and any other
 * operation that reads an existing DEK+ object.
 *
 * May be NULL for:
 * Pure creation operations that do not inspect existing storage,
 * provided LFBT_KEYMGMT_IO_CAP_READ is not advertised.
 *
 * LFBT_keymgmt_write_cb_t write:
 * Callback that copies exactly size bytes from buffer to the serialized
 * DEK+ object, beginning at the DEK+-relative offset.
 *
 * The callback must fail if the backend is read-only, the requested
 * range is invalid, the destination cannot contain the complete
 * transfer, or the backend cannot write all requested bytes. A
 * successful return indicates that all requested bytes were written
 * to the backend, although additional flush or commit processing may
 * still be required before the update is durable.
 *
 * Required for:
 * CREATE, CHANGE_PASSWORD, REMOVE_PASSWORD, ADD_PASSWORD,
 * UPDATE_KDF, MIGRATE, IMPORT_DEK, external-copy creation, and
 * any other operation that creates or rewrites a DEK+ object.
 *
 * May be NULL for:
 * Read-only operations, provided LFBT_KEYMGMT_IO_CAP_WRITE is not
 * advertised.
 *
 * LFBT_keymgmt_get_size_cb_t get_size:
 * Callback that returns, through size_out, the currently available
 * serialized DEK+ object extent, in bytes.
 *
 * For an external DEK+ object, this value normally represents the
 * complete current object size. For an embedded fixed-size backend, it
 * may represent the size of the reserved region available to contain
 * the object. The LFBT_keymgmt implementation must distinguish the
 * backend capacity from the expected serialized object length by
 * validating the DEK+ header and calculated representation size.
 *
 * The callback must fail when size_out is NULL, the size cannot be
 * determined reliably, or the returned value cannot be represented by
 * uint64_t.
 *
 * Required for:
 * Operations that read or validate an existing DEK+ object and
 * operations that must verify destination capacity before writing.
 *
 * LFBT_keymgmt_set_size_cb_t set_size:
 * Callback that changes the storage extent available for the serialized
 * DEK+ object to size bytes.
 *
 * This callback is primarily intended for resizable external storage.
 * A fixed reserved region within an HDF5 file may not require or
 * support resizing and may leave this callback NULL.
 *

* The callback must not truncate or replace an existing object unless
 * the selected operation and runtime arguments authorize the change.
 * When transactional callbacks are in use, resizing performed as part
 * of an update must participate in the same transaction.
 *

* Required for:
 * A write operation whose destination must be resized before the
 * complete serialized DEK+ object can be stored.
 *

* May be NULL for:
 * Fixed-capacity backends or backends whose write implementation
 * manages object sizing internally, provided
 * LFBT_KEYMGMT_IO_CAP_RESIZE is not advertised.
 *

* LFBT_keymgmt_flush_cb_t flush:
 * Callback that requests propagation of completed writes according to
 * the durability guarantees of the selected backend.
 *

* For a native-file backend, this may flush buffered data to the
 * operating system or stable storage. For an embedded VFD backend, it
 * may invoke the applicable VFD or file-layer flush operation. A
 * successful return means that the backend's documented flush
 * guarantee has been satisfied; it does not imply stronger durability
 * than the backend provides.
 *

* Required for:
 * Operations whose policy or backend contract requires an explicit
 * flush before reporting success or committing an update.
 *

* May be NULL for:
 * Backends that do not expose an explicit flush operation,
 * provided LFBT_KEYMGMT_IO_CAP_FLUSH is not advertised and the
 * selected operation does not require it.
 *

* LFBT_keymgmt_begin_update_cb_t begin_update:
 * Optional callback that begins a transactional or atomic replacement
 * operation.
 *

* After this callback succeeds, subsequent write, resize, and flush
 * operations associated with the current key-management request must
 * affect the pending update rather than prematurely replacing the
 * previously valid DEK+ object.
 *

* The backend defines the exact implementation. For example, it may
 * create a temporary external file, begin a journaling operation,
 * allocate a staging buffer, or prepare an alternate reserved region.
 *

* Required for:
 * Operations requesting LFBT_KEYMGMT_OP_ATOMIC_REPLACE when the
 * backend advertises LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE.
 *

* Must be NULL when:
 * The backend does not support transactional updates.
 *

* LFBT_keymgmt_commit_update_cb_t commit_update:
 * Optional callback that commits a previously started transactional
 * update and makes the newly written DEK+ object the active

```

*      representation.
*
*      This callback must be invoked only after begin_update has succeeded
*      and all required serialization, write, resize, and flush operations
*      have completed successfully. A successful return indicates that the
*      replacement has been committed according to the backend's documented
*      atomicity and durability guarantees.
*
*      If commit_update fails, the implementation should invoke
*      abort_update when doing so is valid for the backend and may still
*      prevent exposure of an incomplete replacement.
*
*      Required for:
*          Operations requesting LFBT_KEYMGMT_OP_ATOMIC_REPLACE when the
*          backend advertises LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE.
*
*      Must be NULL when:
*          The backend does not support transactional updates.
*
*      LFBT_keymgmt_abort_update_cb_t abort_update:
*          Optional callback that abandons a transactional update after
*          begin_update has succeeded but before commit_update has completed
*          successfully.
*
*          The callback should discard staged data and preserve or restore the
*          previously valid DEK+ object whenever the backend can do so. It must
*          be safe to invoke during error cleanup after a partial write,
*          failed flush, failed validation, or failed commit attempt.
*
*          The implementation should preserve the original operation error if
*          abort_update also fails, while recording the abort failure as
*          additional diagnostic information when possible.
*
*      Required for:
*          Operations requesting LFBT_KEYMGMT_OP_ATOMIC_REPLACE when the
*          backend advertises LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE.
*
*      Must be NULL when:
*          The backend does not support transactional updates.
*
*      uint64_t capabilities:
*          Bitmask describing the storage operations supported by this backend.
*          Initial capability bits may include:
*
*              LFBT_KEYMGMT_IO_CAP_READ
*              LFBT_KEYMGMT_IO_CAP_WRITE
*              LFBT_KEYMGMT_IO_CAP_RESIZE
*              LFBT_KEYMGMT_IO_CAP_FLUSH
*              LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE
*
*          Capability bits describe backend functionality only. They do not
*          authorize a key-management operation, permit a security-reducing
*          representation, or replace validation of LFBT_keymgmt_args_t.
*
*          Each advertised capability must be consistent with the configured
*          callbacks. For example:

```

```

*           - READ requires a non-NULL read callback.
*           - WRITE requires a non-NULL write callback.
*           - RESIZE requires a non-NULL set_size callback.
*           - FLUSH requires a non-NULL flush callback.
*           - ATOMIC_UPDATE requires non-NULL begin_update, commit_update,
*             and abort_update callbacks.
*
*           Unknown capability bits must be rejected unless a future interface
*           version explicitly defines them as safe to ignore.
*
* Validation Requirements:
*
*           Before beginning an operation, the LFBT_keymgmt implementation must
*           validate at least the following:
*
*           - struct_size is large enough for the supported structure version;
*           - struct_version is supported;
*           - no unknown capability bits are set;
*           - each advertised capability has the required callback;
*           - each required operation capability is advertised;
*           - each callback required by the selected operation is non-NULL;
*           - atomic replacement is rejected unless all transactional callbacks
*             and LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE are present; and
*           - the backend-reported extent can contain the validated serialized
*             representation or can be resized safely when resizing is allowed.
*
*           The implementation must complete this validation before reading secret
*           input, deriving KEK material, recovering a DEK, or modifying persistent
*           storage.
*
* *****/
typedef struct LFBT_keymgmt_io_t {
    size_t    struct_size;
    uint32_t  struct_version;

    void *context;

    LFBT_keymgmt_read_cb_t      read;
    LFBT_keymgmt_write_cb_t     write;
    LFBT_keymgmt_get_size_cb_t  get_size;
    LFBT_keymgmt_set_size_cb_t  set_size;
    LFBT_keymgmt_flush_cb_t     flush;

    LFBT_keymgmt_begin_update_cb_t  begin_update;
    LFBT_keymgmt_commit_update_cb_t commit_update;
    LFBT_keymgmt_abort_update_cb_t  abort_update;

    uint64_t capabilities;
} LFBT_keymgmt_io_t;

```

Initial I/O capability bits may be defined as:

```

#define LFBT_KEYMGMT_IO_CAP_READ      (1ULL << 0)
#define LFBT_KEYMGMT_IO_CAP_WRITE    (1ULL << 1)
#define LFBT_KEYMGMT_IO_CAP_RESIZE   (1ULL << 2)
#define LFBT_KEYMGMT_IO_CAP_FLUSH    (1ULL << 3)

```

```
#define LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE    (1ULL << 4)
```

The `LFBT_keymgmt` implementation must validate the I/O context before beginning an operation. A read-only operation requires `read` and `get_size`. An operation that creates or rewrites a DEK+ object requires `write` and any additional capabilities implied by the selected operation flags. If `LFBT_KEYMGMT_OP_ATOMIC_REPLACE` is requested, the operation must fail unless the backend supplies the necessary transactional callbacks and advertises `LFBT_KEYMGMT_IO_CAP_ATOMIC_UPDATE`. The I/O context is provided separately from `LFBT_keymgmt_args_t`. `LFBT_keymgmt_args_t` describes the requested key management operation and runtime policy; `LFBT_keymgmt_io_t` describes the trusted storage backend selected by the Encryption VFD or companion utility. Keeping these structures separate prevents application-controlled operation arguments from being confused with implementation-controlled storage callbacks.

5.3.7 Internal DEK+ Read/Write Helpers

The read/write helper functions should be implemented as internal routines and should not be exposed as public API entry points. These functions provide the interface between the DEK+ implementation and the serialized on-disk DEK+ object. Several of these helper functions are defined below.

```
herr_t LFBT_keymgmt_dekplus_read(LFBT_keymgmt_io_t *io,
LFBT_dekplus_hdr_t *dekplus_hdr, uint8_t *salt, uint8_t *iv, uint8_t
*dek_payload, uint8_t *mac);
```

Reads the fixed header and payload from the configured DEK+ storage location. Primarily used when working with an existing HDF5 file that requires the Encryption VFD for operations. This helper should:

1. call `get_size`;
2. read and deserialize the fixed header;
3. validate the header before trusting lengths;
4. calculate the expected serialized size safely;
5. compare the expected size against the backend-reported size;
6. read only the fields valid for the selected representation.

```
herr_t LFBT_keymgmt_dekplus_write(LFBT_keymgmt_io_t *io, const
LFBT_dekplus_hdr_t *dekplus_hdr, const uint8_t *salt, const uint8_t
*iv, const uint8_t *dek_payload, const uint8_t *mac);
```

Writes the fixed header and payload to the configured DEK+ storage location. Primarily used during initial DEK+ object creation, KEK rotation, or future key format migration. This helper should:

1. serialize into a temporary ordinary-memory buffer;
2. validate the complete serialized representation before writing;
3. begin a transactional update when requested;
4. write the complete object;

5. flush when supported or required;
6. commit the update;
7. abort on any intermediate failure.

```
herr_t LFBT__keymgmt_dekplus_init_header(LFBT_dekplus_hdr_t
*dekplus_hdr, uint32_t flags);
```

Creates a new key header with the default values:

```
magic           = "EVFDKEY\0"
version         = 1
kdf_id          = GCRY_KDF_SCRIPT
kdf_n           = 32768
kdf_r           = 8
kdf_p           = 1
kdf_out_len     = 64
cipher_id       = GCRY_CIPHER_AES256
cipher_mode     = GCRY_CIPHER_MODE_CBC
mac_id          = HMAC-SHA256 project identifier
dek_len         = 32
salt_len        = 16
iv_len          = 16
dek_payload_len = 48
mac_len         = 32
```

```
herr_t LFBT__keymgmt_dekplus_validate_header(const
LFBT_dekplus_hdr_t *dekplus_hdr);
```

Function used to validate DEK+ object metadata before cryptographic use. Rejects malformed or unsupported DEK+ objects. Validation includes the following:

```
magic
version
flags
KDF parameters
cipher/mode/MAC identifiers
field lengths
expected total file size
```

These helper functions are responsible for reading and writing the fixed-size header and payload fields, encoding and decoding little-endian integer fields, validating field lengths, checking supported format versions, and ensuring that the DEK+ object conforms to the expected Version 1 layout. Because these routines operate on the DEK+ serialized representation, they should process the DEK+ header, salt, IV, wrapped DEK, and authentication tag using ordinary application memory. These values are metadata,

encrypted data, or authenticated data and do not require allocation from the `libgcrypto` secure memory pool.

5.3.8 Internal Cryptographic Helpers

The cryptographic helper functions encapsulate the primitive cryptographic operations required by the local key-generation solution. These routines are responsible for generating cryptographically secure random DEKs, deriving KEK material from user passwords through the selected Key Derivation Function, wrapping and unwrapping the DEK, and computing and verifying the authentication tag associated with the DEK+ object. Isolating these operations behind a well-defined interface simplifies maintenance of the Encryption VFD while ensuring that cryptographic operations remain centralized, consistent, and easier to audit. As the implementation evolves, these helpers also provide a natural location for introducing additional algorithms or cryptographic policies without requiring changes to the higher-level workflow. Representative internal helpers include:

```
herr_t LFBT__keymgmt_generate_dek(uint8_t **out_secure_dek, size_t
dek_len);
```

Allocates `dek_len` bytes from the `libgcrypto` secure memory pool and fills it with random bytes (32 bytes is the assumed size for the initial implementation). Memory allocation should be done using `gcry_malloc_secure(dek_len)` and the random bytes should be generated with `gcry_randomize(..., GCRY_STRONG_RANDOM)`.

```
herr_t LFBT__keymgmt_generate_dekplus_material(uint8_t salt[16],
uint8_t iv[16]);
```

Generates the random cryptographic material required when creating or rewriting a DEK+ object. Specifically, this helper generates a new KDF salt and CBC initialization vector using `libgcrypto`'s cryptographically secure random number generator. The generated values are non-secret and therefore do not require allocation from the `libgcrypto` secure memory pool.

```
herr_t LFBT__keymgmt_derive_kek_material(const char *password, const
uint8_t *salt, size_t salt_len, const LFBT_dekplus_hdr_t
*dekplus_hdr, uint8_t **out_secure_kek);
```

Uses `scrypt` to derive 64 bytes. Of note, `kek_material[0..31]` is the AES-256-CBC wrapping key and `kek_material[32..63]` is the HMAC-SHA256 key.

```
herr_t LFBT__keymgmt_wrap_dek(const uint8_t *secure_dek, const
uint8_t kek_material[64], const uint8_t iv[16], uint8_t
wrapped_dek[48]);
```

Encrypts the 32-byte binary DEK using AES-256-CBC with one block of padding.

```
herr_t LFBT__keymgmt_recover_dek(const uint8_t wrapped_dek[48],
const uint8_t kek_material[64], const uint8_t iv[16], uint8_t
**out_secure_dek);
```

Decrypts and un pads the wrapped DEK. The result must be exactly 32 bytes and should be copied into libgcrypt secure memory.

```
herr_t LFBT__keymgmt_compute_dekplus_mac(const LFBT_dekplus_hdr_t
*dekplus_hdr, const uint8_t *salt, const uint8_t *iv, const uint8_t
*wrapped_dek, uint8_t mac_out[32], const uint8_t kek_material[64]);
```

Computes the HMAC-SHA256 over the authenticated DEK+ object contents.

```
herr_t LFBT__keymgmt_verify_dekplus_mac(const LFBT_dekplus_hdr_t
*dekplus_hdr, const uint8_t *salt, const uint8_t *iv, const uint8_t
*wrapped_dek, const uint8_t expected_mac[32], const uint8_t
kek_material[64]);
```

Recomputes and compares the MAC using a constant-time comparison.

5.3.9 Password and Policy Helpers

The password and policy helper functions implement the runtime security policies associated with password handling and user interaction. These routines validate whether a particular operation is permitted under the current Encryption VFD configuration, determine whether empty-password operation is allowed, and populate the appropriate DEK+ format flags during key creation or KEK rotation. By separating policy enforcement from both the cryptographic operations and the file I/O, these helper functions ensure that password-related decisions remain consistent throughout the implementation and may be modified independently as additional authentication methods or configuration options are introduced.

Terminal presentation and password collection should be implemented through a separate internal prompt interface rather than directly within the password-policy validation helpers. Policy helpers determine whether prompting and the requested password representation are permitted; the prompt interface manages warnings, consent, input collection, confirmation, and temporary-buffer cleanup.

```
herr_t
LFBT__keymgmt_validate_password_policy(LFBT_keymgmt_operation_t
operation, const LFBT_keymgmt_args_t *args,
uint32_t *resulting_dekplus_flags);
```

Enforces the empty-password policy. The rules are as follows:

- On Creation:

- Empty password is allowed only when `allow_empty_password == true`
- On `CHANGE_PASSWORD` or `ADD_PASSWORD`:
 - An empty `new_password` is allowed only when `allow_empty_password == true` and empty-password support is compiled.
- On Unlock:
 - Empty password is allowed only when the DEK+ object has the `LFBT_DEKPLUS_FLAG_EMPTY_PASSWORD` flag set
 - If `LFBT_DEKPLUS_FLAG_CLEARTEXT_DEK` is set, password validation is skipped entirely.

```
bool LFBT__keymgmt_password_is_empty(const char *password);
```

Determines whether the provided password should be treated as an empty-string password. A `NULL` pointer is treated as an invalid programming error and is not considered equivalent to an empty password. Returns `true` only when the provided password points to a valid zero-length C string (`""`).

```
void LFBT__keymgmt_secure_clear(void *buffer, size_t size);
```

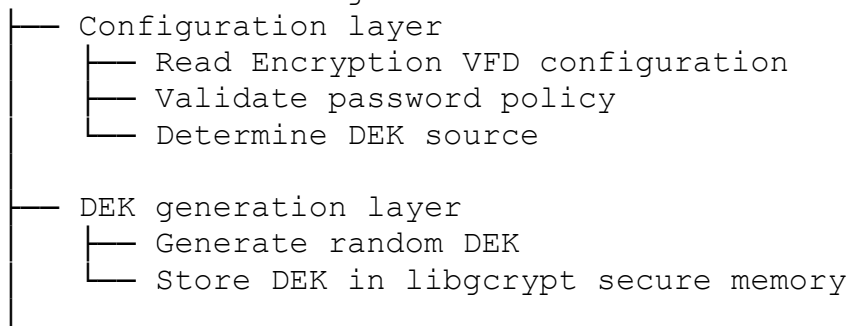
Securely overwrites sensitive memory before it is released or reused. This helper should use an implementation that prevents compiler optimization from removing the memory-clearing operation. It should be used for plaintext password buffers, derived KEK material, temporary plaintext DEK buffers, and any other temporary secret material that exists outside the `libgcrypt` secure memory pool.

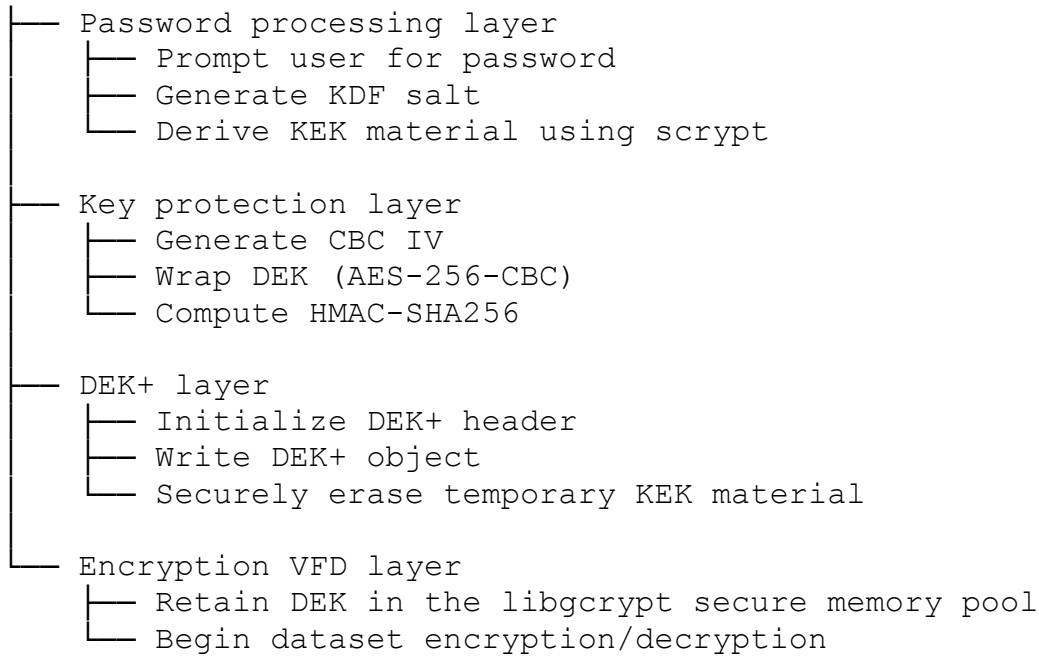
Note: This helper should not be used to release `libgcrypt` secure memory allocations. Secure memory allocated through `libgcrypt` should instead be released using corresponding `libgcrypt` memory management routines.

5.4 Call-Stack Outlines

5.4.1 Creating a New DEK+ Object

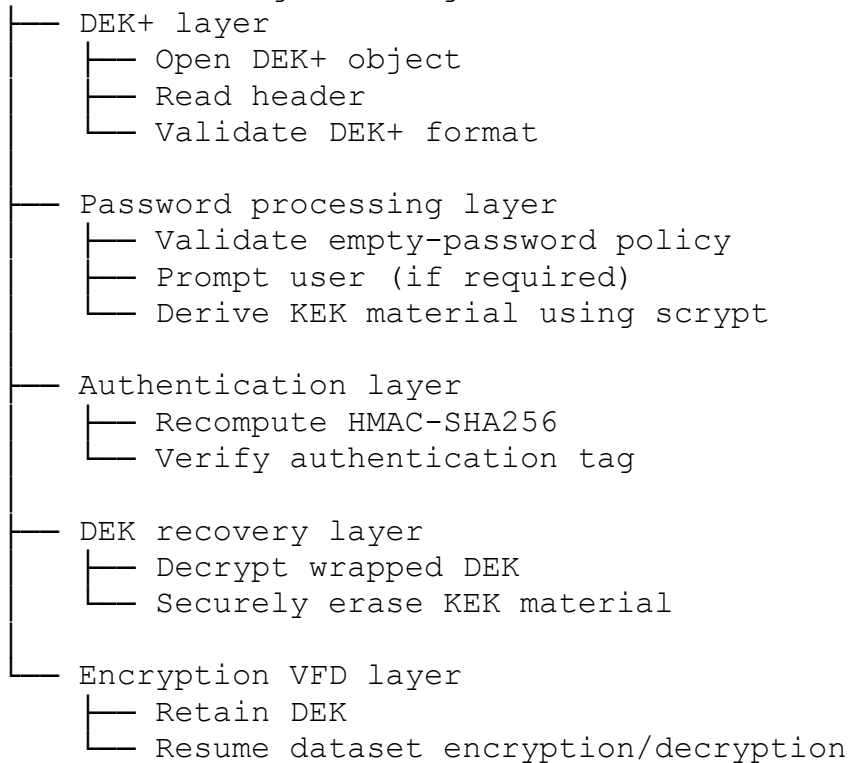
Create New DEK+ Object





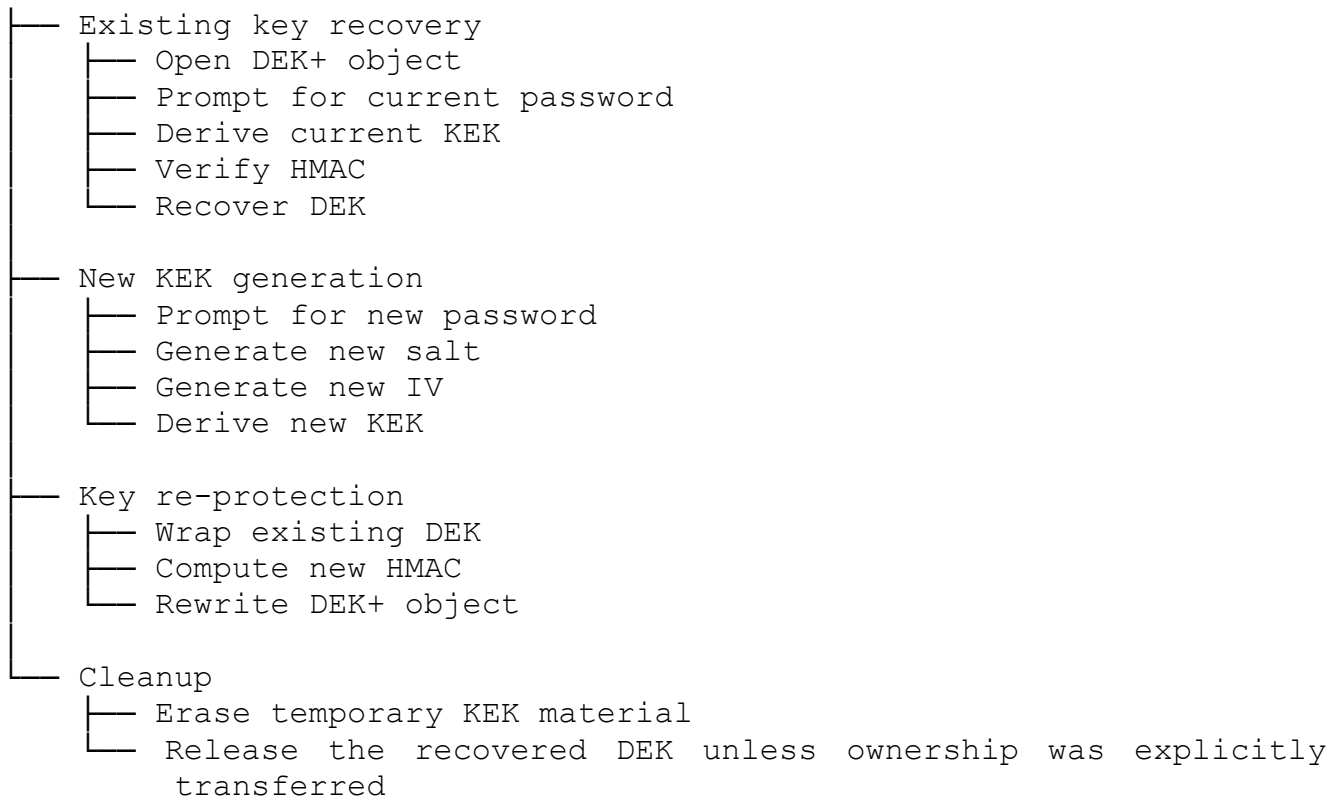
5.4.2 Unlock Existing DEK+ Object

Unlock Existing DEK+ Object



5.4.3 Rekey Workflow (h5keytools) (Password Change/Removal)

Rekey KEK



5.5 Testing Plan

As of the time of writing, the following scenarios should be covered by testing:

- Successful creation of a new DEK+ object.
- Successful recovery of a wrapped DEK.
- Incorrect password.
- Empty-password operation.
- Password-to-password KEK rotation.
- Password-to-empty KEK rotation.
- Empty-to-password KEK rotation.
- Corrupted header.
- Corrupted wrapped DEK.
- Corrupted HMAC.
- Invalid version.
- Invalid KDF parameters.
- Invalid cipher identifiers.

- Missing or inaccessible DEK+ object.
- Secure memory pool cleanup following both success and failure paths.

Appendix A – Glossary

- **Authentication Key:** The second 32 bytes of KEK material derived by the Key Derivation Function. Used exclusively by HMAC-SHA256 to compute and verify the DEK+ authentication tag.
- **Authentication Tag:** A value computed over protected data using a keyed authentication algorithm. In the DEK+ object, the authentication tag is the HMAC-SHA256 output computed over the header, KDF salt, IV, and wrapped DEK. It is used to detect accidental corruption and unauthorized modification before the wrapped DEK is decrypted.
- **Authenticity:** Assurance that the authenticated portions of the DEK+ format were produced or modified only by an entity possessing the correct authentication key derived from the user password.
- **Binary blob:** An opaque sequence of bytes interpreted by the application rather than a standardized file format.
- **Cipher:** A cryptographic algorithm used to transform plaintext into ciphertext, or ciphertext back into plaintext, using a key. In the initial DEK+ implementation, AES-256-CBC is used to wrap and unwrap the DEK.
- **Ciphertext:** Encrypted data produced by applying a cipher to plaintext. In terms of the DEK+ format, the wrapped DEK is considered ciphertext.
- **Cleartext:** Unencrypted data that is directly usable without requiring a decryption operation. In terms of this design, a cleartext DEK refers to a DEK that is provided directly through configuration or stored as an unencrypted binary blob.
- **Data file:** An HDF5 file that has been processed using the Encryption VFD.
- **Data Encryption Key (DEK):** Symmetric key used to encrypt and decrypt HDF5 data using the Encryption VFD.
- **Decrypt:** Generic cryptographic term for the inverse operations used to recover ciphertext into plaintext.
- **DEK+ Object:** A serialized instance conforming to the DEK+ format. In the initial implementation, the object is stored in a reserved page of an encrypted HDF5 file. Future versions may support external DEK+ objects. Pronounced “DEK plus”.
- **DEK+ Format:** The byte-level serialization specification that defines the DEK+ header, payload fields, cryptographic metadata, and supported representations. Pronounced “DEK plus”.

- **DEK+ Representation:** The cryptographic state encoded by a DEK+ object, such as a wrapped DEK representation protected by a non-empty password, a wrapped DEK representation protected by an empty-string password, or a cleartext DEK representation.
- **Deserialize:** The process of reconstructing in-memory data structures from their serialized on-disk representation.
- **Encrypt:** Generic cryptographic term for the operations used to transform plaintext into ciphertext.
- **Encrypt-then-MAC:** A cryptographic construction in which data is first encrypted and then authenticated by computing a message authentication code over the resulting ciphertext and selected metadata. During recovery, authentication is verified before any decryption operation is performed.
- **Encryption VFD:** The HDF5 Virtual File Driver responsible for encrypting and decrypting HDF5 data using a recovered DEK. In this design, the Encryption VFD consumes key material but is not responsible for general-purpose key migration or KEK rotation.
- **HMAC:** A keyed message authentication code constructed from a cryptographic hash function. In the DEK+ object, HMAC-SHA256 is used to authenticate the header, salt, IV, and wrapped DEK.
- **Integrity:** Assurance that the authenticated portions of the DEK+ format have not been modified since the authentication tag was generated.
- **Initialization Vector (IV):** A non-secret value used together with a symmetric encryption algorithm to ensure that identical plaintext encrypted under the same key produces different ciphertext. The IV is required to unwrap the DEK after the KEK material has been derived. A new random IV should be generated each time the DEK is wrapped using AES-256-CBC.
- **Key Derivation Function (KDF):** A cryptographic algorithm that derives one or more secret keys from a password and associated parameters.
- **Key Encryption Key (KEK):** Key derived from the user password and used wrap and unwrap the DEK.
- **libgcrypt:** A general-purpose cryptographic library developed as part of the GnuPG project. Within the Encryption VFD, `libgcrypt` provides the cryptographic primitives required for random-number generation, symmetric encryption, message authentication, password-based key derivation, and secure memory management. This initial implementation relies on `libgcrypt` for AES-256-CBC, HMAC-SHA256, `scrypt`, cryptographically secure random-number generation, and allocation of sensitive cryptographic material within a protected secure memory pool.
- **Nonce:** A value that is intended to be used only once during a cryptographic operation. Depending on the underlying cryptographic algorithm, a nonce often serves a similar role to an IV.

- **Ordinary Application Memory:** Memory allocated using conventional process allocation mechanisms (for example, `malloc()`) rather than a `libgcrypt` secure memory pool. Ordinary application memory should not be used for long-term storage of secret cryptographic material.
- **Plaintext:** Data before encryption or after a successful decryption. In this design, the plaintext DEK refers to the unwrapped 32-byte binary DEK before it is encrypted into the wrapped DEK.
- **Recover / Recovery:** The complete process of making the plaintext DEK available for use. When using a DEK+ object, recovery consists of reading the DEK+ metadata into ordinary process memory, deriving the KEK material from the user password, verifying the authentication tag, decrypting the wrapped DEK, allocating space within the `libgcrypt` secure memory pool, copying the recovered plaintext DEK into the `libgcrypt` secure memory pool, and securely erasing any temporary cryptographic material.
- **Salt:** A randomly generated, non-secret value combined with the user-provided password during Key Derivation Function (KDF) execution. The salt ensures that identical passwords produce different derived KEKs and reduces the effectiveness of attacks based on precomputed password tables, such as rainbow-table attacks. The salt is necessary for recovering the KEK and will be stored as part of the DEK+ format.
- **scrypt:** A Key Derivation Function (KDF) designed to securely derive cryptographic keys from passwords or passphrases. It uses configurable work factors to increase the computational and memory cost of offline password-guessing attacks.
- **Secret Material:** Cryptographic information whose disclosure would compromise the security of the DEK+ object or the protected data. In this design, secret material includes the user password, derived KEK material, and the recovered plaintext DEK. Secret material should be allocated from the `libgcrypt` secure memory pool whenever practical and securely erased when no longer required.
- **Secure Erase:** The process of intentionally overwriting sensitive memory using a method that prevents compiler optimization from removing the overwrite operation.
- **Secure Memory Pool:** A `libgcrypt`-managed region of locked memory used for storing sensitive cryptographic material. Memory allocated from the secure memory pool is intended to resist paging to disk and includes additional protections intended to reducing the likelihood of memory disclosure.
- **Serialize:** The process of converting in-memory data structures into their on-disk binary representations.
- **Serialization Format:** The byte-level representation of the DEK+ header and associated fields as written to persistent storage.
- **Structural Validation:** Verification that a DEK+ object is well formed, supported, and internally consistent without requiring the user password or authentication key.
- **Unwrap:** The decryption operation applied to the wrapped DEK to recover the plaintext 32-byte binary DEK.
- **User Password:** The secret entered by the user to derive the KEK.

- **Wrapped DEK:** The at-rest encrypted form of the DEK stored in the proposed DEK+ object format.
- **Wrapping Key:** The first 32 bytes of KEK material derived by the Key Derivation Function. Used exclusively by AES-256-CBC to wrap and unwrap the DEK.
- **Work Factor:** A collection of parameters that determine the computational and memory cost required to execute a Key Derivation Function. Increasing the work factor increases the cost of both legitimate key recovery and password-guessing attacks.

Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0024823 for Phase I and Phase II SBIR project “Protecting the confidentiality and integrity of data stored in HDF5”

References

1) Carper, Clay (2026). Encryption Key Storage Access Mechanisms. Lifeboat LCC

2) Tadayoshi Kohno, Niels Ferguson, & Schneier, B. (2010). Cryptography engineering: design principles and practical applications. Wiley Pub., Inc.

3) Barker, E. (2020, May 4). Recommendation for Key Management: Part 1 – General. Csrc.nist.gov. <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>

4) Schneier, B. (2015). Applied Cryptography, Second Edition Protocols, Algorithms, and Source Code in C. Indianapolis, Indiana John Wiley & Sons, Inc.

5) OASIS. (2023). “PKCS #11 Specification Version 3.1.” Oasis-Open.org, docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/pkcs11-spec-v3.1.html. Accessed 1 June 2026.

Revision History

July 9, 2026:	Version 1-3 circulated for comment within Lifeboat, LLC.